



SISTEMA DE AUTOMAÇÃO DE INFRAESTRUTURA PARA APLICAÇÕES DE EDGE COMPUTING

Lucas de Queiroz dos Reis

Projeto de Graduação apresentado ao Curso de Engenharia Eletrônica e de Computação da Escola Politécnica, Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Engenheiro.

Orientador: Luís Henrique Maciel Kosmalski
Costa

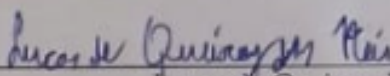
Rio de Janeiro
Setembro de 2024

SISTEMA DE AUTOMAÇÃO DE INFRAESTRUTURA PARA APLICAÇÕES DE EDGE COMPUTING

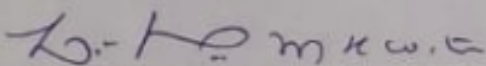
Lucas de Queiroz dos Reis

PROJETO DE GRADUAÇÃO SUBMETIDO AO CORPO DOCENTE DO CURSO
DE ENGENHARIA ELETRÔNICA E DE COMPUTAÇÃO DA ESCOLA PO-
LITÉCNICA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO COMO
PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO GRAU
DE ENGENHEIRO ELETRÔNICO E DE COMPUTAÇÃO

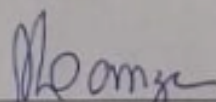
Autor:


Lucas de Queiroz dos Reis

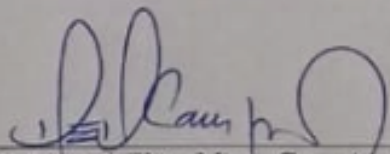
Orientador:


Prof. Luís Henrique Maciel Kosmowski Costa, Dr.

Examinador:


Prof. Marcelo Luiz Drummond Lanza, M.Sc.

Examinador:

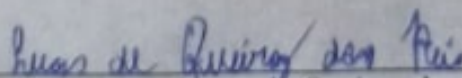

Prof. Miguel Elias Mitre Campista, D.Sc.

Rio de Janeiro
Setembro de 2024

Declaração de Autoria e de Direitos

Eu, *Lucas de Queiroz dos Reis* CPF 107.664.427-99, autor da monografia *Sistema de automação de infraestrutura para aplicações de Edge Computing*, subscrevo para os devidos fins, as seguintes informações:

1. O autor declara que o trabalho apresentado na disciplina de Projeto de Graduação da Escola Politécnica da UFRJ é de sua autoria, sendo original em forma e conteúdo.
2. Excetuam-se do item 1. eventuais transcrições de texto, figuras, tabelas, conceitos e idéias, que identifiquem claramente a fonte original, explicitando as autorizações obtidas dos respectivos proprietários, quando necessárias.
3. O autor permite que a UFRJ, por um prazo indeterminado, efetue em qualquer mídia de divulgação, a publicação do trabalho acadêmico em sua totalidade, ou em parte. Essa autorização não envolve ônus de qualquer natureza à UFRJ, ou aos seus representantes.
4. O autor pode, excepcionalmente, encaminhar à Comissão de Projeto de Graduação, a não divulgação do material, por um prazo máximo de 01 (um) ano, improrrogável, a contar da data de defesa, desde que o pedido seja justificado, e solicitado antecipadamente, por escrito, à Congregação da Escola Politécnica.
5. O autor declara, ainda, ter a capacidade jurídica para a prática do presente ato, assim como ter conhecimento do teor da presente Declaração, estando ciente das sanções e punições legais, no que tange a cópia parcial, ou total, de obra intelectual, o que se configura como violação do direito autoral previsto no Código Penal Brasileiro no art.184 e art.299, bem como na Lei 9.610.
6. O autor é o único responsável pelo conteúdo apresentado nos trabalhos acadêmicos publicados, não cabendo à UFRJ, aos seus representantes, ou ao(s) orientador(es), qualquer responsabilização/ indenização nesse sentido.
7. Por ser verdade, firmo a presente declaração.



Lucas de Queiroz dos Reis

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

Escola Politécnica - Departamento de Eletrônica e de Computação

Centro de Tecnologia, bloco H, sala H-217, Cidade Universitária

Rio de Janeiro - RJ CEP 21949-900

Este exemplar é de propriedade da Universidade Federal do Rio de Janeiro, que poderá incluí-lo em base de dados, armazenar em computador, microfilmear ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es).

DEDICATÓRIA

Ao meu pai, Osvaldo Cristiano dos Reis (In memoriam).

À minha mãe, Sueli Alves de Queiroz.

A todos aqueles que acreditam no desenvolvimento tecnológico para tornar o mundo melhor.

AGRADECIMENTO

A jornada através do curso de Engenharia Eletrônica e de Computação não foi uma tarefa fácil. Muito esforço foi necessário para esta conclusão. Após me formar como técnico em eletrônica no CEFET-RJ, considerei que o caminho natural fosse me tornar engenheiro. Doce ilusão de que o curso técnico teria me preparado para o que estava por vir.

Esta história provavelmente já deve ter se repetido diversas vezes, exceto na parte de que fiz parte da seleção brasileira de powerlifting. O atraso e a interrupção da minha graduação para focar em um esporte competitivo foi algo que nem eu poderia ter previsto. Insegurança, dúvida e incapacidade de gerir o tempo foram sensações que me acompanharam ao longo de metade da minha graduação. Muitas vezes me peguei pensando em como uma graduação pode ser tão difícil, ao ponto de me questionar se ser campeão mundial não seria mais fácil.

Além de minha mãe Sueli e de minha esposa Thay, tive a felicidade de encontrar diversos amigos ao longo do caminho que apoiaram o meu jeito duro de tentar resolver as coisas. Respondendo às perguntas deles sobre minha graduação com frases como “A UFRJ não é difícil, eu que sou fraco”, “Eu nunca tive burnout” e outras, enquanto realizava uma dezena de certificações em desenvolvimento de software ao longo dos períodos, sempre tive o apoio de todos que compartilhavam da minha visão de repetir sempre a frase “estudar muito para não trabalhar para banco a vida inteira”, “chegar a um lugar onde eu possa desenvolver soluções que façam a vida das pessoas melhor, e se não fizer melhor, que pelo menos seja mais barata”. Um notório agradecimento aos meus queridos amigos Rafael de Andrade, Luiza Cavalcante, Rafael Hilst, Lucas Fidalgo, Anna Letícia Alegria, João Pedro Castelo Branco e Laís Hasek. A todos vocês, muito obrigado.

Um agradecimento também àqueles que não vão ler este projeto, mas também não estão ausentes, que são os meus três gatos Fred, Nescau e Naru. Obrigado pela companhia nas centenas de horas de estudo madrugada adentro.

Gostaria também de agradecer à minha empresa atual Petrec - Petróleo e Pesquisa, e em especial ao meu supervisor de estágio Raphael Vilela pela porta aberta para o perfil DevOps e para o desenvolvimento de diversas tecnologias e responsabilidades. A premissa original deste projeto foi sendo afinada e curiosamente foi sendo definida baseada em desenvolver um TCC onde eu pudesse treinar tecnologias que ainda não estivesse utilizando no trabalho e que pudessem vir a ser úteis no futuro próximo.

Por último e não menos importante, gostaria de agradecer em especial ao professor Marcelo Luiz Drumond Lanza e ao meu orientador de projeto de graduação Luís Henrique Maciel Kosmowski Costa, não só pelo conhecimento, mas também pela paciência com as minhas muitas perguntas. Muito obrigado.

De toda maneira, a jornada para me tornar um engenheiro me deixou muito feliz ao compreender a visão holística adquirida durante as disciplinas. Visualizar áreas como eletrônica, computação e controle para projetar soluções que resolvem problemas no mercado de trabalho pode não ser exatamente um lazer, porém com imenso prazer digo que fazer este curso valeu muito a pena.

A todos aqueles que não foram citados, mas fizeram parte da minha jornada acadêmica, não só como engenheiro, mas também como um ser humano melhor, deixo aqui meu muito obrigado.

RESUMO

O desenvolvimento de tecnologias da Indústria 4.0 envolve a criação de sistemas "vivos", que operam continuamente, mesmo durante atualizações, utilizando ferramentas de automação de infraestrutura de software. DevOps, como metodologia, integra desenvolvimento e operações para promover colaboração, automação e entrega contínua de software, sendo amplamente adotada na computação em nuvem. Em ambientes de nuvem híbrida, que combinam nuvem pública e infraestrutura local *bare-metal* na borda, o balanceamento de serviços e cargas entre esses ambientes torna-se essencial. Neste cenário, pipelines de Integração Contínua e Entrega Contínua (CI/CD) desempenham um papel crucial ao automatizar processos de construção, teste e implantação de software, assegurando uma entrega ágil e confiável de novas funcionalidades e atualizações. A adoção de práticas DevOps e a implementação de pipelines CI/CD permitem que as organizações maximizem o uso eficiente da computação em nuvem e infraestrutura local, assegurando operações escaláveis e eficazes. Este projeto define uma prova de conceito para um sistema IoT integrado à nuvem, utilizando a metodologia DevOps para desenvolver um pipeline CI/CD que possibilita a atualização de uma solução de borda a partir da nuvem. A pesquisa abrange o uso de ferramentas de infraestrutura *open-source* para integração *Cloud-Edge-IoT* e aplicação de aprendizado de máquina (ML) em problemas específicos. Como exemplo prático, foi implementada uma "ambulância inteligente", onde o Jenkins, uma plataforma de automação de código aberto para CI/CD, permite atualizações automatizadas de software em hardware embarcado em veículos em movimento, sem necessidade de manutenção manual. Além do uso do Jenkins, foram criados scripts Linux para funções gerais e, com base nas heurísticas de testes do Jenkins, estabeleceram-se tempos de referência para cada etapa dos testes de atualização de código.

Palavras-Chave: computação na nuvem, computação na borda, integração contínua, entrega contínua.

ABSTRACT

The development of Industry 4.0 technologies involves creating "living" systems that operate continuously, even during updates, using software infrastructure automation tools. DevOps, as methodology, integrates development and operations to promote collaboration, automation, and continuous delivery of software, widely adopted in cloud computing. In hybrid cloud environments, which combine public cloud and local *bare-metal* infrastructure at the edge, balancing services and workloads across these environments becomes essential. In this context, Continuous Integration and Continuous Deployment (CI/CD) pipelines play a crucial role by automating software building, testing, and deployment processes, ensuring fast and reliable delivery of new features and updates. Adopting DevOps practices and implementing CI/CD pipelines allow organizations to maximize the efficient use of cloud computing and local infrastructure, ensuring scalable and effective operations. This project defines a proof of concept for an IoT system integrated with the cloud, using the DevOps methodology to develop a CI/CD pipeline that enables updating an edge solution from the cloud. The research includes the use of *open-source* infrastructure tools for *Cloud-Edge-IoT* integration and applying machine learning (ML) to specific problems. As a practical example, an "intelligent ambulance" was implemented, where Jenkins, an open-source automation platform for CI/CD, allows automated software updates on hardware embedded in moving vehicles without manual maintenance. In addition to using Jenkins, Linux scripts were created for general purposes, and based on Jenkins' test heuristics, reference times were established for each stage of the code update tests.

Keywords: cloud computing, edge computing, continuous integration, continuous delivery.

SIGLAS

API - *Application Programming Interface*

AWS - *Amazon Web Services*

CD - *Continuous Deployment*

CI - *Continuous Integration*

DSL - *Domain Specific Language*

E2E - *End to End*

GPS - *Global Positioning System*

HTTP - *Hypertext Transfer Protocol*

IA - *Artificial Intelligence (AI)*

IAM - *Identity and Access Management*

IoT - *Internet of Things*

IP - *Internet Protocol*

JSON - *JavaScript Object Notation*

JWT - *JSON Web Token*

MQTT - *Message Queuing Telemetry Transport*

OS - *Operating System*

RHEL - *Red Hat Enterprise Linux*

SCM - *Source Control Management*

SSH - *Secure Shell*

SSL - *Secure Sockets Layer*

TI - *Tecnologia da Informação*

TLS - *Transport Layer Security*

VM - *Virtual Machine*

Sumário

1	Introdução	1
1.1	Delimitação	3
1.2	Objetivos	3
1.3	Metodologia	4
1.4	Organização do Texto	5
2	Fundamentação Teórica	6
2.1	DevOps	6
2.2	Git e CI/CD	9
2.3	Contêineres e Orquestradores	10
2.4	Provisionamento, VMs e Configuração	15
2.5	Monitoramento e Observabilidade	16
2.6	CI/CD com Jenkins para Ambientes de Borda	18
2.7	Plataformas de integração nuvem e borda	20
2.8	Nuvem híbrida	21
2.9	Ferramentas Adotadas	21
3	Prova de Conceito	23
3.1	A Aplicação	24
3.2	A Infraestrutura	28
4	Conclusões	39
	Bibliografia	43
A	Código-fonte da prova de conceito	46

Lista de Figuras

1.1	Diagrama ilustrando modelo do sistema a ser implementado.	5
2.1	Exemplo de Ciclo DevOps. Fonte: 4Linux [1].	7
2.2	Exemplo de pipeline Jenkins interativo. Fonte: [2].	10
2.3	Exemplo de arquitetura KubeEdge. Fonte: KubeEdge [3].	12
2.4	Exemplo de arquitetura K3. Fonte: K3s [4].	13
3.1	Hierarquia das requisições dos módulos.	24
3.2	Página Web do módulo da ambulância.	27
3.3	Pipelines executados com sucesso.	29
3.4	Pipeline falhou ao testar o processamento de áudio para inalação de fumaça.	33
3.5	Pipeline 91 falhou ao testar o processamento de áudio para inalação de fumaça.	34
3.6	Pipeline 90 executado com sucesso fazendo o deploy enquanto importa as novas imagens.	36

Lista de Tabelas

2.1	Organização dos Repositórios do Projeto.	22
3.1	Estrutura do Projeto.	23
3.2	Funções dos módulos da aplicação.	25
3.3	Módulos de Processamento de Áudio.	26
3.4	Tabela de contêineres Docker do projeto.	28
3.5	Resultados das Execuções de Pipeline na figura 3.3 (Transposta e reduzida).	30
3.6	Etapas do Pipeline CI/CD do Jenkins.	30
3.7	Resultados das Execuções de Pipeline na Figura 3.4 (Transposta). . .	33

Capítulo 1

Introdução

A criação de produtos que conectam dispositivos físicos à Internet, conhecida como Internet das Coisas (IoT – *Internet of Things*) , tem gerado um aumento significativo no tráfego de dados a cada ano. Em contraste, a flexibilidade da computação em nuvem trouxe novos desafios. Isso culminou na necessidade de sistemas distribuídos que balanceiem a nuvem e a borda da rede, a fim de servir o cliente final.

A adoção de “sistemas inteligentes” e Inteligência Artificial (IA)—tecnologias capazes de realizar tarefas que normalmente requerem inteligência humana—tem sido fundamental para otimizar processos da vida cotidiana. Esse avanço é impulsionado pelo desenvolvimento das telecomunicações, como o 5G, que permite o desenvolvimento de hardware na borda para reduzir a latência. Além disso, esses sistemas facilitam a integração de dispositivos IoT, possibilitando o mapeamento de dados analógicos para a rede de forma eficiente.

As relações trabalhistas dentro do mercado de Engenharia de Computação utilizam Metodologias Ágeis como facilitador no desenvolvimento destas aplicações. Desta forma este trabalho estuda métodos de automatizar o desenvolvimento e implantação de um produto computacional que se encontre neste nicho.

Este trabalho desenvolve uma prova de conceito de um pipeline CI/CD executado pela nuvem como facilitador do desenvolvimento de produtos IoT atualizáveis sendo executados na borda da nuvem.

O sistema consiste em pesquisar ferramentas DevOps para decidir quais serão utilizadas no desenvolvimento deste projeto.

Será construído um pipeline de CI/CD que, a partir de um gatilho de evento disparado pela atualização de código em um repositório no GitHub, iniciará um processo de criação de imagens dos serviços a serem testados.

Serão utilizados diversos métodos de tradução de áudio para texto. Ao comparar a qualidade das traduções na API de processamento de voz do quinto repositório, o pipeline poderá decidir se utilizará ou não aquele código. Dessa forma, serão criadas soluções que passem em todos os testes estabelecidos, assim como soluções que falhem em alguns testes, para demonstrar a tomada de decisão automatizada pelo pipeline.

A aplicação desenvolvida conta com APIs construídas utilizando o framework FastAPI [5]. Este é um framework para a criação de APIs em Python. Ele permite um desenvolvimento, com validação automática de dados e suporte para funcionalidades assíncronas. Além disso, oferece a geração automática de documentação.

Para simular serviços paralelos na arquitetura proposta, foi utilizada uma interface em Django [6]. O Django é um framework web de alto nível para Python, que facilita o desenvolvimento rápido e seguro de aplicações web. Ele foi utilizado para exemplificar o produto central, que se comunica com diversos módulos.

Além disso, foi usado o NginX [7] como Proxy Reverso. O NginX serve como um API Gateway hipotético dentro da rede. Para reconhecimento de voz, foram utilizadas algumas bibliotecas como SpeechRecognition [8], PyTorch [9], e Librosa [10].

Além disso, o Flask [11] é utilizado para desenvolver APIs leves e flexíveis. Flask é um microframework de web para Python que fornece as ferramentas necessárias para criar rapidamente aplicações web e APIs RESTful. No contexto deste projeto, Flask é empregado para construir uma API de tradução de voz para texto, permitindo testar e integrar múltiplos métodos de reconhecimento de voz.

Ferramentas para controle de versão, containerização, testes e implantação como Git[12], Docker[13], Docker-compose, Jenkins[2] e scripts livres foram utilizados para desenvolver e entender os prós e contras de diferentes regras de negócio.

Desta forma, foi criado um modelo de aplicação web que exemplifica o conceito de uma ambulância que recebe voz e geolocalização de um paramédico e a partir de tal entra em contato com um hospital. Desta forma podendo tomar uma decisão

de orientar através da geolocalização o endereço do hospital que estatisticamente tem a maior chance de salvar o paciente.

Essa ideia ilustra o cenário de milhares de dispositivos semelhantes, em constante movimento e inacessíveis para manutenção direta. Esses dispositivos dependem de atualizações de software que precisam ser geridas e testadas de forma automática. A implementação de pipelines CI/CD simplifica o processo de atualização, permitindo a implantação de novos softwares sem a necessidade de intervenções manuais.

1.1 Delimitação

A delimitação deste estudo se concentra no desenvolvimento de uma infraestrutura em nuvem que permita testar e atualizar softwares sendo executados na borda que controlam dispositivos IoT e que executam módulos de aprendizado de máquina. A ênfase será na análise e projeto de sistemas de automação de infraestrutura para a arquitetura “nuvem mais ambulância” proposta considerando os requisitos de escalabilidade e desempenho.

1.2 Objetivos

O objetivo é portanto desenvolver uma aplicação parcial de maneira que esta utilize um pipeline CI/CD para acelerar o processo de implantação da mesma. A criação de pequenas aplicações que traduzam áudio para texto como ponto de apoio para definir quais características deverão ser aprovadas ou reprovadas a partir da verificação se a tradução está semanticamente aceitável utilizando expressões regulares. A criação deste pipeline e a definição de políticas são o foco central deste estudo. O objetivo é que essas políticas não atendam apenas a esta aplicação, mas também outros sistemas distribuídos entre a nuvem e a borda. O modelo de ambulância inteligente serve como exemplo para essa análise. O cenário visado é aquele no qual seja possível verificar uma atualização sendo implementada corretamente além da re-provação de implantações defeituosas em milhares de ambulâncias simultaneamente sem que estas se tornem inoperantes.

1.3 Metodologia

O projeto será conduzido em várias etapas para desenvolver a prova de conceito de uma ambulância inteligente. Esta ambulância utiliza módulos de geolocalização e voz para executar modelos de aprendizado de máquina na borda. A comunicação com hospitais, que possuem serviços em nuvem, permite processar a voz do paramédico e convertê-la em texto. Em seguida, esses dados são cruzados com informações de geolocalização. Após consultar as APIs dos hospitais na nuvem, a ambulância recebe uma sugestão da melhor rota hospitalar a ser seguida. Dessa forma, o projeto examina como realizar atualizações dessa aplicação por meio de um pipeline CI/CD, permitindo a atualização automática de milhares de ambulâncias com base em políticas de software.

1. Estudo de Ferramentas de desenvolvimento de infraestrutura:

- Realizar um estudo detalhado das ferramentas necessárias para a implementação do projeto como Git, Docker, Ansible, Terraform e Jenkins.

2. Desenvolvimento da Aplicação:

- Criar a aplicação proposta e implementar os módulos de geolocalização e voz, que serão responsáveis por tomar decisões com base nos modelos de aprendizado de máquina.

3. Infraestrutura e pipeline CI/CD:

- Utilizar ferramentas que satisfaçam a criação de um pipeline CI/CD de forma que a infraestrutura proposta para a borda seja efetivamente automatizada.

A Figura 1.1 ilustra a solução proposta por este trabalho, que envolve o desenvolvimento de uma parte do software destinada à execução na nuvem e outra na borda, além das aplicações necessárias para essa integração.

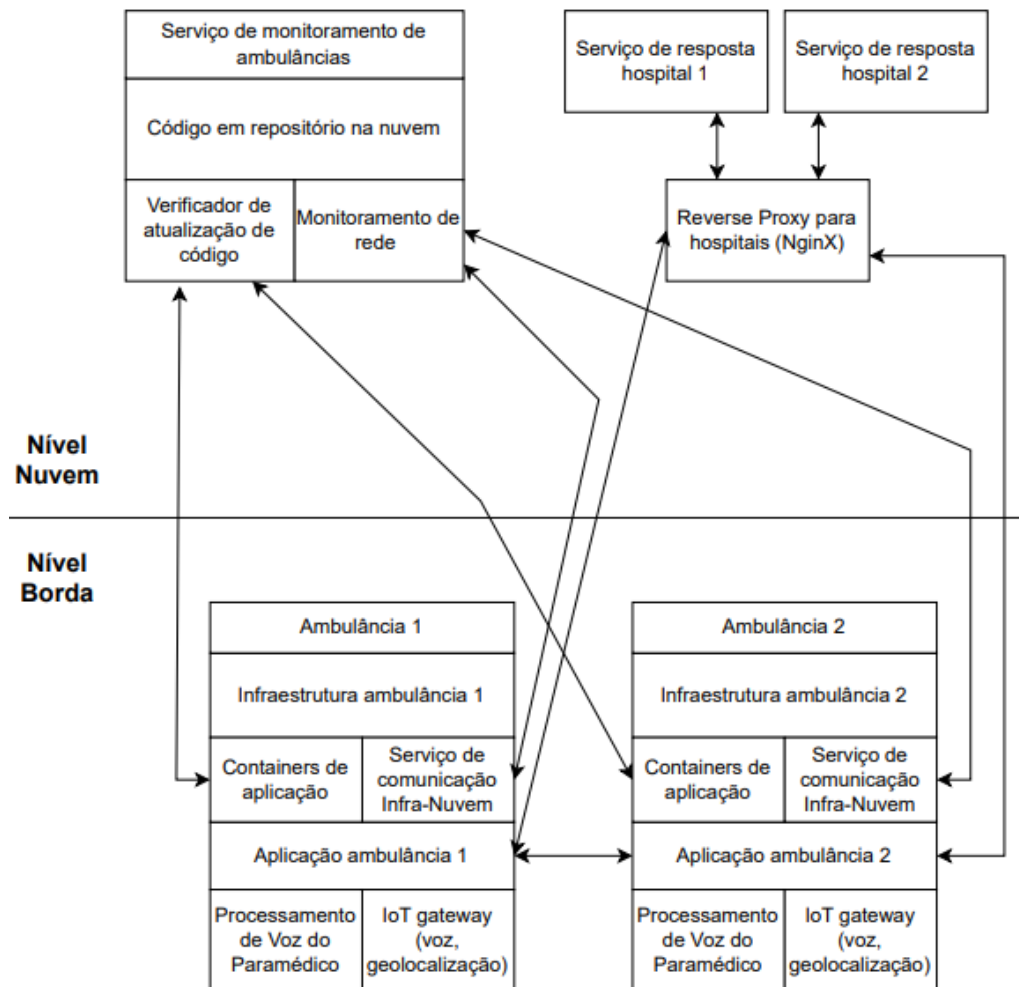


Figura 1.1: Diagrama ilustrando modelo do sistema a ser implementado.

1.4 Organização do Texto

No Capítulo 2, apresentaremos a fundamentação teórica por meio da pesquisa de tipos de ferramentas que auxiliam na automação de infraestrutura de TI e determinaremos quais serão utilizadas no projeto, juntamente com trabalhos relacionados de soluções já propostas por desenvolvedores e empresas, a fim de inspirar o projeto. A prova de conceito será apresentada no Capítulo 3. Neste capítulo será detalhada a solução proposta, justificando as decisões de projeto a partir dos testes fundamentados durante a produção do Capítulo 2. O Capítulo 4 conclui o trabalho e apresenta direções de desenvolvimento futuras.

Capítulo 2

Fundamentação Teórica

Este capítulo introduzirá os conceitos necessários para entender DevOps, seus objetivos e métodos de alcançá-los.

2.1 DevOps

O DevOps define uma metodologia de desenvolvimento de software focada em estabelecer uma infraestrutura direcionada ao ciclo de vida do software. [14]

Ao desenvolver um código e prepará-lo para produção, especialmente em ambientes de DevOps, é essencial seguir diversas etapas para garantir a qualidade e a eficiência do software. Essas etapas incluem versionamento do código, testes automatizados, monitoramento contínuo, configuração de ambientes, e virtualização. Essas práticas fazem parte do ciclo de integração contínua (CI) e entrega contínua (CD), que são fundamentais para a automação de infraestrutura e a agilidade no desenvolvimento.

Quando se trata de executar esse código na nuvem para disponibilizar um produto online, como um site ou uma API pública, entram em cena ferramentas e práticas específicas para a automação da infraestrutura em nuvem. Nesse contexto, é crucial conhecer tópicos e ferramentas populares que suportam essa automação, como gerenciamento de contêineres, orquestração de serviços, e monitoramento de desempenho em tempo real.

A Figura 2.1 referencia o ciclo DevOps de CI e CD como uma repetição infinita de etapas para integração e entrega de novo código.

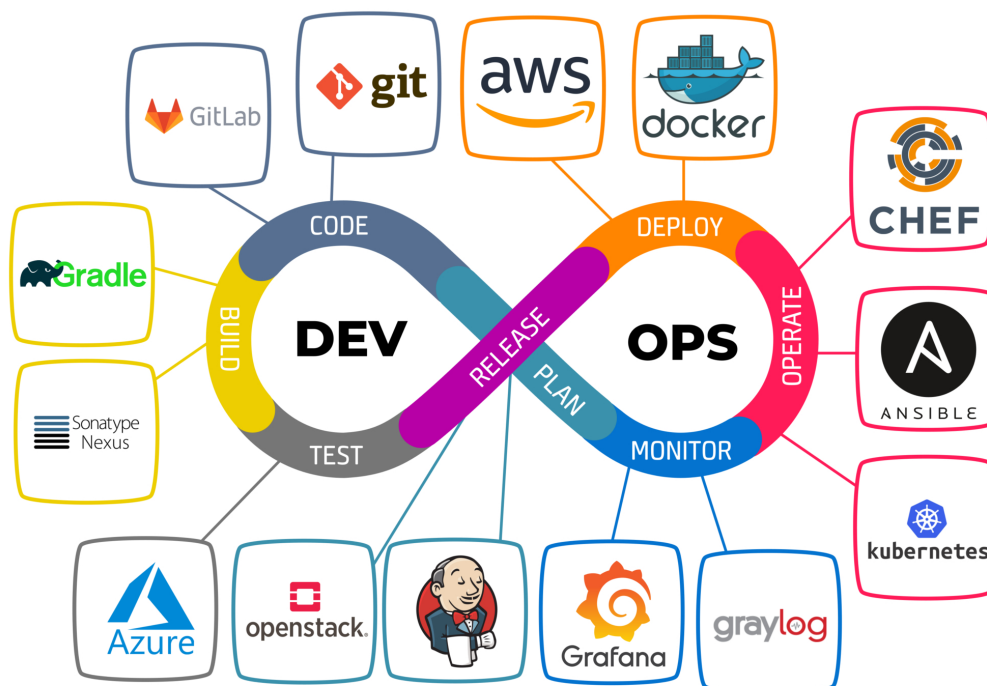


Figura 2.1: Exemplo de Ciclo DevOps. Fonte: 4Linux [1].

1. **Code (Codificação):** Nesta fase, o código é escrito e versionado. Ferramentas como **Git** e **GitLab** são utilizadas para controlar as versões do código, permitindo a colaboração entre desenvolvedores e o rastreamento de mudanças.
2. **Build (Construção):** O código é compilado ou preparado para execução. Ferramentas como **Gradle** e **Sonatype Nexus** ajudam na automação do processo de construção e na gestão de dependências.
3. **Test (Testes):** O código é testado para garantir que ele funcione corretamente. Ferramentas como **Jenkins** são usadas para automatizar os testes, integrando-os ao pipeline de CI/CD.
4. **Release (Liberação):** Após os testes, o código é preparado para ser lançado. Esta fase é onde a preparação para a implantação ocorre, normalmente integrando processos de revisão e aprovação.
5. **Deploy (Implantação):** O código é implantado nos ambientes de produção ou pré-produção. Ferramentas como **AWS** e **Docker** são usadas para gerenciar a

infraestrutura e os containers que suportam a aplicação.

6. Operate (Operação): Nesta fase, o software é monitorado enquanto está em operação, garantindo que ele funcione conforme esperado. Ferramentas como **Chef**, **Ansible** e **Kubernetes** auxiliam na gestão da configuração e da orquestração dos serviços.

7. Monitor (Monitoramento): O desempenho do software é monitorado continuamente durante a operação a fim de coletar dados para identificar e resolver problemas. Os problemas encontrados nesta etapa serão planejados para serem resolvido na etapa de codificação do próximo ciclo. Ferramentas como **Grafana** e **Graylog** são usadas para visualização e análise de logs e métricas de desempenho.

A ideia por trás da infraestrutura como código (IaC – Infrastructure as Code) é que você escreva e execute código para definir, implantar, atualizar e destruir a infraestrutura. Isso representa uma mudança importante de mentalidade na qual se trata todos os aspectos das operações como software, mesmo aqueles aspectos que representam hardware (por exemplo, configuração de servidores físicos). Na verdade, uma percepção fundamental do DevOps é que você pode gerenciar quase tudo em código, incluindo servidores, bancos de dados, redes, arquivos de log, configuração de aplicativos, documentação, testes automatizados, processos de implantação, e assim por diante.

Existem cinco categorias amplas de ferramentas IaC, cada uma desempenhando um papel específico na automação e gestão da infraestrutura. Scripts ad hoc são usados para automação simples e tarefas repetitivas, geralmente escritos em linguagens de script como Bash ou Python. Ferramentas de gerenciamento de configuração são usadas para garantir que os sistemas permaneçam em um estado desejado, automatizando a aplicação de configurações consistentes em múltiplos servidores. Ferramentas de modelagem de servidores, que permitem criar ambientes de desenvolvimento que replicam a produção, facilitando o teste e o desenvolvimento local. Ferramentas de orquestração, que são usadas para gerenciar contêineres e coordenar o funcionamento de aplicações distribuídas em larga escala. Por fim, ferramentas de provisionamento, que permitem criar e gerenciar infraestrutura em diferentes provedores de nuvem de forma declarativa e reutilizável.

Ao entender essas categorias, é possível selecionar a combinação certa de ferramentas para atender às necessidades específicas de uma organização, garantindo que a infraestrutura seja gerenciada de forma eficiente, escalável e confiável.

2.2 Git e CI/CD

O Git [15] é a ferramenta de controle de versão mais utilizada da atualidade. Sendo esta composta por uma relação entre o servidor que armazena e gerencia o código e clientes via Interface de Linha de Comando (CLI – Command Line Interface) e Interface Gráfica do Usuário (GUI – Graphical User Interface). Características como armazenar revisão, criar linhas do tempo alternativas em clientes locais e no servidor e a possibilidade de fundir estas linhas estão entre as principais tarefas do Git.

A integração contínua é o processo de integrar novos códigos escritos pelos desenvolvedores com um branch principal ou “master” frequentemente ao longo do dia. Após a integração de códigos de diferentes origens, ocorre a geração de novos executáveis e a realização de testes com o novo código.

Integração Contínua é a substituição automática do código em caso de sucesso nos testes executados a partir dos executáveis construídos pelo novo código. Todas estas etapas serem automatizadas cria uma experiência de desenvolvimento mais eficiente permitindo um lançamento superior de atualizações para aplicações online.

Ferramentas que criam pipelines de construção, testes e entrega de atualização personalizada para cada caso fazem parte das tarefas de uma plataforma em nuvem. Todos estes conceitos também existem na computação em borda. Considerando o caso de estudo sendo uma nuvem híbrida com um servidor físico conectado à nuvem, o código pode tanto ser armazenado em nuvem por motivos de segurança e enviar apenas os executáveis necessários para o servidor físico ou todas as instruções podem ser feitas pelo próprio servidor físico.

Plataformas como o GitHub e GitLab são utilizadas como plataformas de hospedagem de código fonte. Estas possuem métodos de comunicação e autenticação que intermedeiam as relações de compartilhamento de código em um ambiente onde desenvolvedores de um mesmo projeto não estejam em constante contato.

Um serviço popular para criação de pipelines para CI/CD é o Jenkins [2]. Este é um administrador de builds open-source customizável através de plugins para interagir com outras ferramentas e que possui uma Linguagem Específica de Domínio (DSL – Domain-Specific Language). assim como uma GUI para construção de pipelines. A DSL do Jenkins é escrita em Groovy [16] e permite a descrição de um pipeline e suas instruções em arquivos “jenkinsfile”. Uma característica importante é a possibilidade de haver pipelines paralelos a fim de criar casos de sucesso que possam existir em apenas um único fluxo gráfico como funcionar apenas em um navegador específico.

A Figura 2.2 mostra um exemplo gráfico de um pipeline com alternativas paralelas de testes, uma vez que o pipeline encerra em caso de erro serial porém o processamento concorrente é permitido.



Figura 2.2: Exemplo de pipeline Jenkins interativo. Fonte: [2].

Um caso interessante para decidir o tipo de gerenciador de pipelines que você vai utilizar é considerar se este é um serviço exclusivo da nuvem como API ou se este é implantável em uma máquina específica. O Jenkins como caso utiliza uma arquitetura que permite criar diversos nós para verificar quando o repositório foi atualizado e tentar uma atualização, enquanto outro nó recebe comunicação e começa alguma ação como gerar testes. Desta forma a distribuição da cadeia de servidores de teste, homologação e produção pode ser configurada como uma reação em cadeia acionada a partir de uma sequência de ações.

2.3 Contêineres e Orquestradores

Quando o assunto é automação de infraestrutura, a padronização de ambiente é um dos tópicos mais importantes. O ambiente de testes precisa ser o mais próximo

possível do ambiente de produção a fim de que o workflow criado pelo gerenciador de integração contínua possa impedir bugs com maior eficácia possível. Para resolver isso utilizamos contêineres: o menor encapsulamento possível para executar imagens sem um sistema operacional no core do Linux. A tecnologia mais utilizada como contêiner é o Docker [13].

Para se criar regras detalhadas das relações entre criação e comunicação de contêineres diferentes utiliza-se plataformas orquestradoras. Orquestradores são gerenciadores do fluxo de trabalho de múltiplos contêineres e como estes se comunicam com outros serviços.

O serviço mais popular de orquestração é o Kubernetes [17]. Este serviço foi criado pela Google e existem muitas variações dele criadas pelos provedores de nuvem. Entre os orquestradores mais importantes para computação na borda focaremos no K3s e no KubeEdge.

Ambas as variantes do Kubernetes, K3s e KubeEdge, são focadas em IoT e borda. O K3s é uma distribuição leve do Kubernetes que possui uma arquitetura servidor-agente, onde o servidor gerencia o controle centralizado e os agentes são responsáveis por executar as cargas de trabalho. Essa arquitetura é ideal para ambientes com recursos limitados, como dispositivos de borda. No contexto de IoT, o K3s pode ser integrado com brokers MQTT para facilitar a comunicação entre dispositivos IoT, permitindo a troca eficiente de mensagens entre os dispositivos e a infraestrutura central.

O MQTT [18] (Message Queuing Telemetry Transport) é um protocolo de mensagens leve, projetado para comunicação assíncrona e eficiente em redes com baixa largura de banda ou alta latência. Ele é amplamente utilizado em cenários de IoT por sua capacidade de transportar dados de sensores e dispositivos de borda de maneira eficiente para sistemas centrais, permitindo a troca de informações em tempo real, mesmo em condições de conectividade instável.

Já o KubeEdge expande o Kubernetes para suportar o ambiente de borda, dividindo sua arquitetura em níveis Cloud e Edge. No nível Edge, além de suporte a contêineres, o KubeEdge também oferece integração nativa com um broker MQTT, o que facilita a conectividade e a gestão de dispositivos IoT diretamente na borda, sem depender de uma conexão constante com a nuvem.

Essa explicação mais detalhada ajuda a esclarecer o funcionamento e as vantagens de K3s e KubeEdge, especialmente no contexto de IoT e edge computing, ao mesmo tempo que introduz o papel crucial do MQTT como um protocolo de comunicação eficiente.

Estas variantes do Kubernetes habilitam a possibilidade de aproveitar uma melhor experiência de desenvolvimento e automação de infraestrutura comumente vista na nuvem em hardwares de baixa capacidade de processamento na borda.

As Figuras 2.3 e 2.4 referenciam as arquiteturas de K3s e KubeEdge, sendo KubeEdge focado na integração do nível nuvem e nível borda criando elasticidade de contêineres apenas na borda enquanto o K3s utiliza arquitetura de nó principal e secundários para criar elasticidade de contêineres tanto na nuvem quanto na borda.

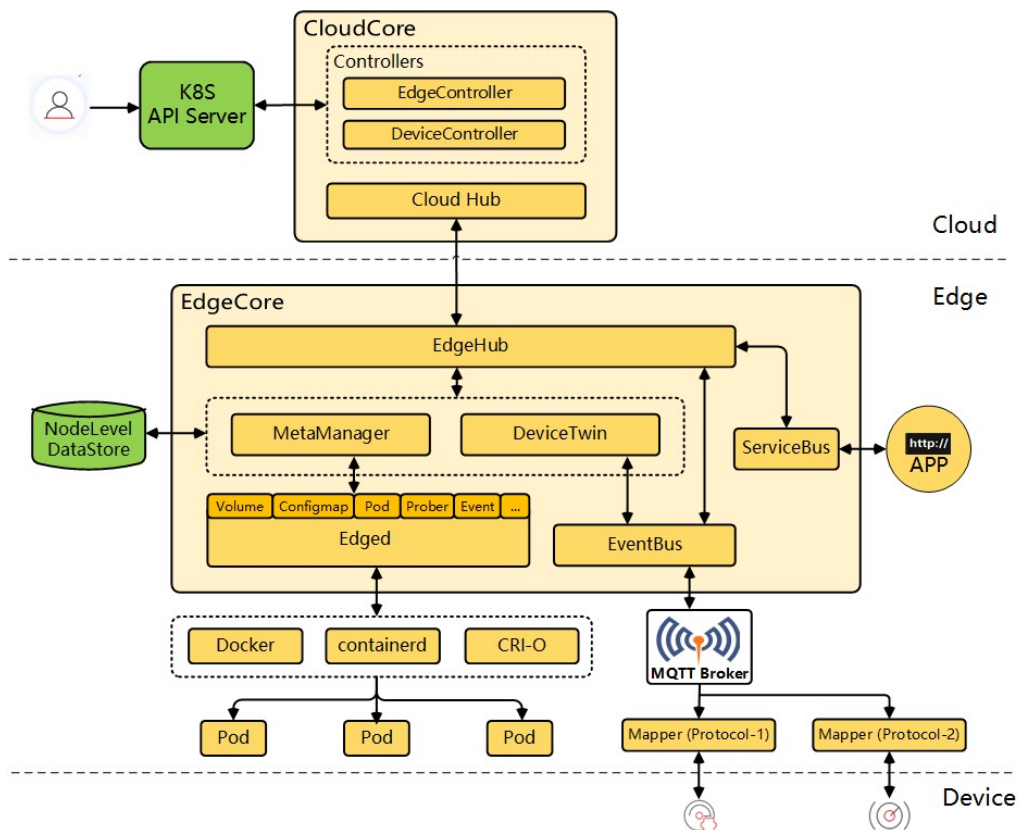


Figura 2.3: Exemplo de arquitetura KubeEdge. Fonte: KubeEdge [3].

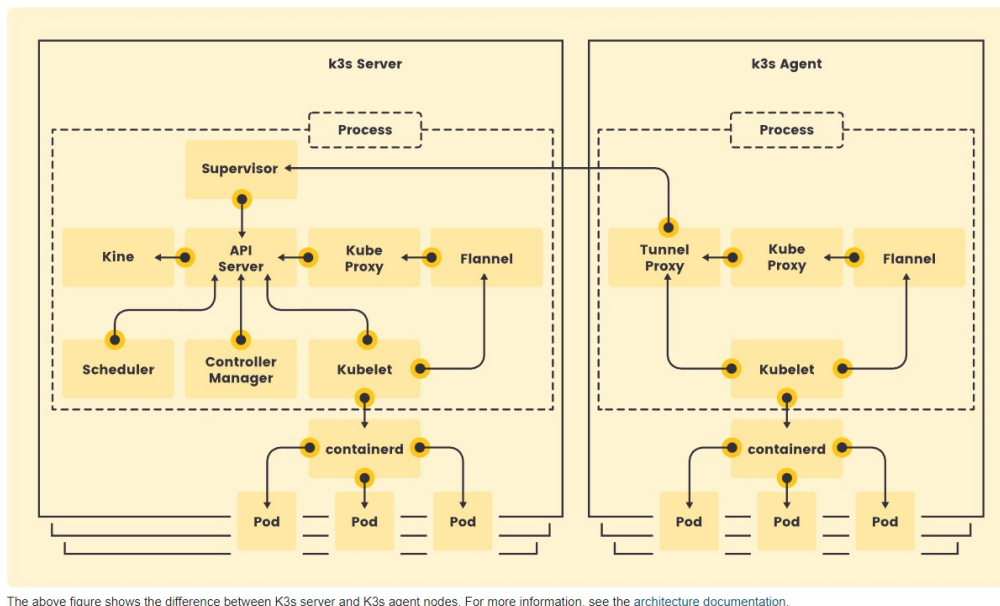


Figura 2.4: Exemplo de arquitetura K3. Fonte: K3s [4].

- **K3s Server:**

- **API Server:** Centraliza as interações entre os componentes, gerenciando as solicitações dos clientes e das outras partes do sistema.
- **Scheduler:** Responsável por designar Pods para os nós disponíveis com base nos recursos e requisitos.
- **Controller Manager:** Gerencia os controladores que respondem às alterações no estado do cluster.
- **Kine:** Um banco de dados leve que substitui o **etcd**, que é um sistema de armazenamento distribuído chave-valor utilizado para armazenar de forma confiável o estado de um cluster Kubernetes.
- **Kube Proxy:** Gerencia as regras de rede que permitem a comunicação entre os Pods.
- **Flannel:** Uma rede de sobreposição usada para conectar Pods em diferentes nós.
- **Kubelet:** Agente responsável por aplicar e monitorar as especificações do Pod no nó.

- **K3s Agent:**

- **Tunnel Proxy:** Facilita a comunicação entre o K3s Server e os agentes.
- **Kube Proxy e Flannel:** Similar ao servidor, eles gerenciam a conectividade de rede nos nós de agente.
- **Kubelet:** Responsável por garantir que os contêineres estejam executando como esperado.
- **containerd:** Gerencia a execução dos contêineres, assegurando que os Pods estejam funcionando corretamente.

- **CloudCore:**

- **K8S API Server:** Facilita a comunicação entre os componentes da nuvem e da borda.
- **EdgeController:** Gerencia a conectividade e a sincronização de dados entre a nuvem e a borda.
- **DeviceController:** Gerencia dispositivos IoT conectados à borda.
- **Cloud Hub:** Centraliza a comunicação entre os componentes do Kube-Edge na nuvem.

- **EdgeCore:**

- **EdgeHub:** Responsável por manter a conectividade entre a nuvem e a borda.
- **MetaManager:** Gerencia a persistência e sincronização de dados na borda, incluindo informações sobre Pods, volumes e configurações.
- **DeviceTwin:** Representa o estado virtual de um dispositivo IoT, facilitando a sincronização e controle.
- **ServiceBus:** Conecta serviços na borda, facilitando a comunicação entre aplicativos e dispositivos.
- **EventBus:** Gerencia a comunicação entre diferentes componentes no nível da borda.
- **MQTT Broker:** Facilita a comunicação entre dispositivos IoT e serviços na borda usando o protocolo MQTT.

- **Contêineres e Pods:**

- **Docker, containerd, CRI-O:** São os runtimes de contêineres que executam os Pods na borda.
- **Pods:** Agrupam os contêineres que executam as aplicações e serviços necessários.

2.4 Provisionamento, VMs e Configuração

Provisionamento de infraestrutura de software é a automação do processo de criação e configuração de infraestrutura através de ferramentas como VMs e scripts.

Em um provedor de nuvem, diversas ferramentas já são providas pela própria nuvem como contêineres, orquestradores e monitoramento. Para estes casos existem formas de escrever a infraestrutura como código em DSLs como o Terraform [19]. O Terraform faz chamadas de API em seu nome para um ou mais provedores de nuvem escritas em um arquivo de configuração. Isto é possível devido à integração open-source da Hashicorp (empresa que produz o Terraform) com diversos provedores de nuvem.

Em casos como o de nuvem híbrida abordado neste projeto, não se pode usar o Terraform a nível de borda porque está trabalhando fora da nuvem. Desta forma, há outras ferramentas de automação de infraestrutura que podem ser utilizadas como é o caso do Ansible: uma ferramenta criada pela RedHat com o intuito de gerenciar scripts Python que podem se comunicar com contêineres como o Docker via SSH e automatizar tanto o servidor físico quanto imagens de contêineres.

Outra limitação de flexibilidade da nuvem híbrida abordada é o fato de cada hardware físico dispor apenas de um OS. Container engines funcionam sobre um único OS e devido a isto há uma limitação ao se precisar prover características de OSs distintos.

Ferramentas de VM como o Vagrant [20] são utilizadas para criar e gerenciar máquinas virtuais, que são ambientes isolados que simulam um computador completo dentro de outro sistema operacional. Essas máquinas virtuais permitem a execução de diferentes sistemas operacionais e aplicativos em um único host físico,

proporcionando flexibilidade e isolamento para testes, desenvolvimento e implantação de software.

VMs resolvem problemas de infraestrutura ao permitir a portabilidade que contêineres não permitem. Apesar de muito mais lentos e pesados que contêineres estes provêm um ambiente computacional inteiro sobre uma máquina e podem resolver problemas de configuração específicos de maneira robusta.

2.5 Monitoramento e Observabilidade

O monitoramento de infraestrutura é uma tarefa importante e normalmente consiste de coletar métricas pré definidas através de logs. Observabilidade é um conjunto de técnicas e ferramentas que permitem a uma equipe DevOps monitorar, diagnosticar e depurar uma infraestrutura de forma eficiente, facilitando a identificação e resolução de problemas em sistemas complexos.

Esta área não será muito abordada uma vez que de caso a caso esta é a que haverá mais variações baseadas nos requisitos da aplicação.

Há três aspectos importantes relacionados ao monitoramento de software: monitoramento de infraestrutura, de aplicação e gerenciamento de logs [21].

O monitoramento de infraestrutura está relacionado a acompanhar eventos e alertar caso ocorra algum erro inesperado, como, por exemplo, uma parte do provedor de nuvem ficar offline. Há diversas ferramentas que combinam monitoramento e observabilidade utilizando alarmes e métricas, como o Datadog, Prometheus, e Grafana, focadas em gerar métricas em tempo real.

O Datadog [22] é uma plataforma de monitoramento e análise que integra dados de servidores, bancos de dados, ferramentas e serviços para fornecer visibilidade completa do desempenho das aplicações e infraestrutura. Ele oferece recursos avançados de monitoramento de logs, métricas e rastreamento distribuído em tempo real.

O Prometheus [23] é um sistema de monitoramento de código aberto que coleta e armazena métricas em série temporal, permitindo a configuração de alertas e a visualização de dados. Ele é amplamente utilizado em ambientes Kubernetes devido à sua integração nativa com o ecossistema de contêineres.

O Grafana [24] é uma plataforma de visualização de código aberto que permite a criação de painéis interativos para monitorar métricas e logs. Ele pode ser integrado com várias fontes de dados, incluindo Prometheus, para fornecer insights visuais sobre o desempenho da infraestrutura.

O monitoramento de aplicação refere-se a observar falhas de execução de código e auxiliar no debug automaticamente, identificando, por exemplo, qual linha e status de erro ocorreram em um serviço contêiner antes de ser reiniciado devido ao erro. Ferramentas como Jaeger, OpenTelemetry, e New Relic analisam o comportamento do software para facilitar o debugging E2E (end-to-end) de uma aplicação.

O Jaeger [25] é uma ferramenta de código aberto para rastreamento distribuído que ajuda a monitorar e solucionar problemas em microserviços, fornecendo visibilidade detalhada sobre o fluxo de solicitações entre serviços, o tempo de resposta e os gargalos.

O OpenTelemetry [26] é uma coleção de APIs, bibliotecas e agentes que fornecem um framework unificado para gerar, coletar e exportar dados de telemetria como rastreamentos, métricas e logs de aplicações distribuídas. Ele é compatível com diversos backends e facilita o monitoramento e rastreamento de aplicações em escala.

O New Relic [27] é uma plataforma de observabilidade que fornece monitoramento em tempo real de aplicações, oferecendo insights detalhados sobre o desempenho do código, rastreamento de erros, e análise de logs. Ele é especialmente útil para identificar e resolver problemas em ambientes complexos de microserviços.

O gerenciamento de logs é a tarefa de processar e armazenar os registros gerados tanto pela aplicação quanto pela infraestrutura, permitindo que estes forneçam métricas ao longo do tempo através de consultas e possibilitem auditorias futuras por meio de relatórios de falhas. Esse gerenciamento inclui a coleta, armazenamento e análise dos dados provenientes dos logs para facilitar a resolução de problemas.

O Elasticsearch [28] é um mecanismo de busca e análise distribuído e de código aberto, capaz de armazenar e pesquisar grandes volumes de dados em tempo real de forma eficiente. Ele permite realizar consultas rápidas e complexas sobre os dados de log coletados.

O Logstash é uma ferramenta de processamento de pipeline que coleta, processa e transforma dados de diversas fontes simultaneamente, preparando-os para

serem armazenados e analisados no Elasticsearch. Ele suporta uma variedade de formatos de dados e oferece flexibilidade na filtragem e enriquecimento dos logs [29].

O Kibana é uma plataforma de visualização que permite explorar e visualizar dados armazenados no Elasticsearch através de dashboards interativos e personalizáveis. Com o Kibana, é possível criar gráficos, mapas e outros tipos de visualizações que auxiliam na identificação de padrões e anomalias nos dados de log [30].

2.6 CI/CD com Jenkins para Ambientes de Borda

A implementação de pipelines de Integração Contínua e Entrega Contínua (CI/CD) com Jenkins para ambientes de borda pode trazer vários benefícios, especialmente em comparação com soluções comerciais, como a AWS IoT. Jenkins, sendo uma plataforma de automação open-source, permite que as organizações construam, testem e implementem aplicativos de maneira eficiente, mesmo em ambientes distribuídos que envolvem dispositivos de borda.

John Marinton Reni, da OptiSol Solutions [31], destaca que o uso de Jenkins para CI/CD em ambientes de borda apresenta um custo menor em comparação com a AWS IoT, além de facilitar a interconexão entre diferentes serviços de nuvem e ser de fácil gerenciamento. No entanto, ele também aponta uma limitação significativa: a restrição de IPs públicos disponíveis para dispositivos IoT, um problema que pode ser mitigado com o uso de técnicas como o encaminhamento de porta.

O AWS IoT é um serviço de nuvem oferecido pela Amazon Web Services que permite que dispositivos IoT se conectem, interajam e troquem dados de maneira segura com outros dispositivos e serviços de nuvem. Ele oferece uma infraestrutura escalável e segura para gerenciar redes de dispositivos IoT, mas pode ter um custo elevado dependendo do volume e das necessidades específicas de integração [32].

Raghu Ram Meda [33] enfoca que regras de firewall, roteamento de tráfego entre instâncias da nuvem, credenciais de nuvem e SSH durante a criação de pipelines E2E são essenciais para a configuração ideal em produção.

Durante a criação do projeto, utilizamos a execução local de todas as partes e mapeamos o `socket` do Docker para que o contêiner do Jenkins pudesse executar contêineres fora do próprio contêiner, ou seja, no nível da máquina. Também foram

criadas redes específicas entre contêineres para que estes pudessem se comunicar sem autenticação.

É válido destacar que, para evoluir de uma prova de conceito ou protótipo para um produto entregável, é fundamental implementar um nível adequado de segurança. Isso envolve o uso de chaves de API, tokens de autenticação, e certificação SSL/TLS.

As chaves de API são credenciais únicas geradas para identificar e autenticar solicitações feitas por aplicativos ao acessar serviços ou APIs (Interfaces de Programação de Aplicações). Elas funcionam como uma camada de segurança que permite controlar e monitorar o acesso aos recursos da API, prevenindo o uso não autorizado.

Os tokens de autenticação, como OAuth e JWT (JSON Web Tokens), são métodos usados para garantir que apenas usuários ou serviços autenticados possam acessar determinados recursos. O OAuth é um protocolo de autorização que permite que aplicativos acessem recursos em nome de um usuário, sem expor suas credenciais. Por exemplo, um usuário pode permitir que um aplicativo acesse suas fotos armazenadas em um serviço de nuvem sem compartilhar sua senha. Já o JWT é um padrão de token que transmite informações seguras entre duas partes, como um cliente e um servidor, de forma compacta e verificável. Ele é comumente usado em sistemas de autenticação para assegurar que a sessão de um usuário esteja autenticada e válida.

Por fim, certificação SSL/TLS refere-se a protocolos criptográficos que garantem a segurança das comunicações na internet. SSL (Secure Sockets Layer) e seu sucessor TLS (Transport Layer Security) protegem os dados transmitidos entre um navegador e um servidor web, impedindo que informações sensíveis, como senhas e dados pessoais, sejam interceptadas por terceiros. A utilização de certificados SSL/TLS é essencial para qualquer aplicação que manuseie dados confidenciais, proporcionando confiança e segurança aos usuários.

Implementar essas medidas é crucial para garantir que o produto final seja seguro, confiável e pronto para o uso em ambientes de produção.

Considerando que o pipeline de testes de módulos provavelmente estará na nuvem, será necessário configurar perfis IAM (Identity and Access Management)

para definir os acessos permitidos na nuvem para o nó/agente Jenkins que executará essa porção do pipeline.

O IAM (Identity and Access Management) é um serviço que permite gerenciar de forma segura as permissões e o acesso a recursos de nuvem. Com perfis IAM, você pode definir exatamente quais recursos um usuário, serviço ou nó/agente tem permissão para acessar, garantindo que cada entidade na nuvem tenha apenas os privilégios necessários para realizar suas funções. Isso é especialmente importante em um ambiente de CI/CD com Jenkins, onde o nó/agente que executa o pipeline na nuvem precisa ter acesso controlado aos recursos, como armazenamento, bases de dados, e outros serviços, para minimizar riscos de segurança.

2.7 Plataformas de integração nuvem e borda

A integração nuvem-borda (edge computing) permite que o processamento de dados ocorra tanto na borda da rede, próximo aos dispositivos IoT, quanto na nuvem, que oferece escalabilidade adicional. Essa combinação garante eficiência e segurança, aproveitando a baixa latência da borda e os recursos amplos da nuvem. As ferramentas descritas, como perfis IAM e pipelines CI/CD com Jenkins, são essenciais para gerenciar essa integração, garantindo controle de acesso, segurança e automação dos processos.

A **Red Hat Ansible Automation Platform** [34] utiliza a pilha de tecnologias da própria Red Hat, como o RHEL [35] (Red Hat Enterprise Linux) e o MicroShift, uma distribuição leve do Kubernetes baseada no OpenShift, projetada para ambientes de borda.

A AWS oferece o AWS IoT GreenGrass [36] que estende as capacidades de nuvem para dispositivos locais, permitindo que eles ajam localmente nos dados que geram, enquanto ainda usam a nuvem para gerenciamento, análise e armazenamento durável. Essencialmente, ele permite a implantação e execução de funções do AWS Lambda em dispositivos de borda, permitindo que eles realizem cálculos, processem dados e interajam com outros dispositivos localmente, mesmo quando não estão conectados à internet.

Desta forma, todas os provedores de nuvem do mercado de **Big Tech** tem

uma solução paga e plataforma robusta que estende as capacidades da nuvem para uma rede local. A implantação em si de soluções específicas pode ser uma opção de barateamento do produto conforme uma aplicação cresce em número de usuários. Porém é válido frisar que a flexibilidade dos produtos disponibilizados continua sendo uma possibilidade de investimento inicial.

2.8 Nuvem híbrida

A computação em nuvem híbrida refere-se a uma abordagem que combina recursos de nuvem pública e privada em uma única infraestrutura de TI. Nesse modelo, as organizações podem usar uma combinação de serviços de nuvem pública, fornecidos por provedores como AWS, Microsoft Azure ou Google Cloud, juntamente com recursos de nuvem privada mantidos internamente.

O projeto da Canonical Openstack [37] abre portas não só para a possibilidade de integração nuvem e borda, mas para a possibilidade da sua borda ser uma nuvem privada. Considerando cenários como o servidor principal de um provedor de nuvem grande como a AWS sendo São Paulo pode-se reduzir custos e latência construindo uma nuvem privada no Rio de Janeiro afim de balancear o tráfego de dados.

Desta forma, um cenário onde se há um pipeline CI/CD Jenkins como o abordado neste projeto pode ser implantado na nuvem privada que também faz parte da rede da borda.

2.9 Ferramentas Adotadas

Para este projeto, foram adotadas várias ferramentas de automação para facilitar o desenvolvimento e a manutenção da infraestrutura. Decidiu-se utilizar scripts *Ad Hoc* para tarefas específicas, como automatizar comandos Docker e atualizar o repositório GitHub de todos os submódulos simultaneamente, garantindo que todas as partes do projeto estejam sempre sincronizadas e atualizadas.

O controle de versão será gerenciado utilizando o Git. Em conjunto com o GitHub, que hospeda os repositórios do projeto, o Git facilita a gestão de branches e a integração contínua de novas funcionalidades ao código base.

O pipeline de CI/CD será desenvolvido utilizando o Jenkins. Para isso, um arquivo `Jenkinsfile` será incluído no repositório principal `TCC_Major_Repo`. Este arquivo definirá o pipeline de construção, testes e implantação, automatizando o fluxo de trabalho e garantindo que todas as etapas sejam executadas de maneira consistente.

Além disso, serão exploradas outras opções como o Ansible, uma ferramenta de automação de TI, para a criação de configurações adicionais tanto no ambiente local quanto dentro de contêineres.

O trabalho será organizado em cinco repositórios, conforme descrito na Tabela 2.1.

Tabela 2.1: Organização dos Repositórios do Projeto.

Repositório	Descrição
TCC_Major_Repo	Contém arquivos de infraestrutura e inclui os outros 4 repositórios como submódulos.
TCC_Ambulance_Interface	Interface da ambulância desenvolvida em Django.
TCC_Dummy_GPS_API	API falsa de GPS, desenvolvida utilizando FastAPI.
TCC_NginX_API_Aggregator	Agregador intermediário de múltiplas requisições, utilizando FastAPI como API e NginX como Proxy.
TCC_Voice_Processing	API desenvolvida em Flask para tradução de áudio para texto.

O código-fonte e a documentação deste projeto estão disponíveis no repositório público do GitHub: https://github.com/QIRoss/TCC_Major_Repo/tree/main.

Capítulo 3

Prova de Conceito

A aplicação foi dividida em 5 módulos, sendo alguns os principais para o desenvolvimento deste projeto o módulo do Jenkins e do Processador de Voz. Os módulos de GPS Falso e Agregador NginX existem apenas para exemplificar no caso da ambulância inteligente onde o resto do produto se conecta.

Tabela 3.1: Estrutura do Projeto.

Módulo	Função
Jenkins	Nó mestre que controla o Docker daemon e verifica atualizações conforme notificado pelo GitHub Token para iniciar o pipeline.
Interface Django	Servidor Django que gera a página web da interface. Envia requisições POST ao Processador de Voz com o áudio a ser descrito, além de realizar requisições GET.
GPS Falso	API que não foi totalmente desenvolvida. Envia um valor estático de coordenadas e um timestamp dinâmico.
Agregador NginX	API que não foi totalmente desenvolvida. O NginX se conecta ao servidor FastAPI e devolve três dados estáticos hipotéticos.
Processador de Voz	API Flask que recebe um áudio através de uma requisição POST e devolve o áudio transcrito.

A estrutura deste projeto conta com um servidor feito em Django que serve uma página Web com espaço para gravar áudio. Ao gravar este áudio e submeter o envio ao servidor Django o mesmo faz requisições para o GPS falso, o Agregador NginX e o Processador de Voz e devolve a resposta na página Web.

Esta aplicação não foi muito desenvolvida uma vez que o propósito desta prova de conceito é abordar a manutenção de uma infraestrutura de software na borda da rede. Os testes foram realizados utilizando o Processador de Voz porque este foi considerado um ponto crítico para o sistema uma vez que caso uma transcrição de áudio seja feita erroneamente as decisões da ambulância irão falhar. Pensando neste conceito foram criadas soluções diferentes de qualidade de transcrição de áudio para que algumas tenham sucesso e outras não. O fracasso é

Estes testes que serão realizados pelo Jenkins testarão diretamente a resposta do processador de voz para áudios pré gravados e não utilizará a interface web nos testes.

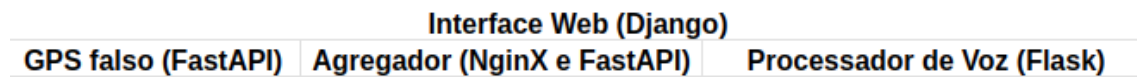


Figura 3.1: Hierarquia das requisições dos módulos.

A estrutura do repositório para inicializar a aplicação e a infraestrutura foi feita utilizando Docker, Docker-Compose e scripts Ad Hoc. Imagens para cada contêiner são criadas para iniciar a aplicação e como volume direto de disco foi utilizado apenas o volume `jenkins_home` para guardar configurações do Jenkins feitas através da interface gráfica como criação de senha, credenciais, configurações específicas da política de cada pipeline como agendamento de consulta do repositório do GitHub para iniciar um pipeline e afins.

O código desenvolvido encontra-se no apêndice deste projeto, assim como no repositório público do GitHub: https://github.com/QIRoss/TCC_Major_Repo.

3.1 A Aplicação

A aplicação é uma ambulância com um hardware linux executando contêineres Docker para gerar interface gráfica web, uma API para dados de GPS, uma API para Processamento de Voz e um agregador de consultas a APIs de hospitais que toma decisões cruzando dados da API de GPS e das consultas.

Esta aplicação se baseia em uma API fictícia para pegar dados de GPS no casos de integração com geolocalização.

A aplicação foi desenvolvida de forma enxuta e desacoplada, permitindo que os testes sejam realizados de maneira independente para cada componente da infraestrutura. Todos estes módulos além da infraestrutura estão no GitHub do autor [38].

Os módulos da aplicação foram construídos usando tecnologias diferentes como incentivo ao entendimento de como a construção de imagens Docker funcionam a fim de otimizar individualmente cada uma delas.

A Tabela 3.2 descreve a função de cada módulo da aplicação.

Tabela 3.2: Funções dos módulos da aplicação.

Módulo	Função
API GPS fictícia (FastAPI)	Envia uma coordenada constante e timestamp.
API Processador de Voz (Flask)	Recebe arquivo de áudio e devolve em texto.
Agregador NginX e FastAPI	Simula um API gateway na nuvem respondendo.
Interface da Ambulância (Django)	Interface Web que recebe as 3 APIs acima e gera uma resposta.

A API GPS fictícia é uma API escrita utilizando FastAPI executando na porta 2947 TCP, porta comum para GPSs. Uma vez que esta prova de conceito foi criada com foco em automação de infraestrutura, não utilizaremos a API GPS fictícia durante os testes de integração porque os testes com a troca dos módulos processadores de voz são mais interessantes para avaliar o sucesso e fracasso de um pipeline CI/CD se baseando na qualidade dos processamentos.

A API Processadora de Voz é uma API Flask que recebe um arquivo de áudio formato WAV e responde um com um JSON equivalente a este em texto. Este módulo além de contar com alguns arquivos de áudio para exemplo de testes unitários conta com 3 arquivos diferentes utilizando métodos diferentes de resolução de áudio.

A Tabela 3.3 define as diferenças em cada uma das 3 versões de processamento de áudio.

Cada uma destas versões de Processamento de Voz são testadas separada-

Tabela 3.3: Módulos de Processamento de Áudio.

Módulo	Função
Google API	Envia para uma API do Google reconhecer via internet.
PocketSphinx	Canal de reconhecimento de voz 16 bits.
Torch, Librosa e transformers (HuggingFace)	Utiliza modelo público de Aprendizado de Máquina pré-treinado.

mente pelo pipeline criado na infraestrutura e descrito mais à frente. Desta forma, podemos exemplificar um método de aprovar ou reprovar atualizações de códigos e dependências como modelos pré-treinados a fim de gerir gestão ideal. Para facilitar a identificação todos os áudios e testes foram feitos em inglês.

O agregador NginX FastAPI é a porta de entrada delegada para o API Gateway do nível nuvem receber resposta dos hospitais.

A página web gerada pela plataforma Django utilizada como interface exemplo da ambulância possui um botão de gravar que permite o envio de um áudio, um botão de stop para encerrar a gravação e um botão de enviar a gravação. A interface se comunica com os módulos API GPS fictícia, API Processador de Voz e Agregador NginX e FastAPI para responder na página a concatenação das mensagens das 3 APIs.

A Figura 3.2 mostra a página Web do módulo da ambulância recebendo uma transcrição de um texto em inglês depois de o mesmo ter sido captado pelo microfone. Também pode-se observar as coordenadas de latitude e longitude junto com um timestamp vindo da API do GPS. A terceira resposta vem do NginX que representa a nuvem agregando 3 dados diferentes de 3 possíveis hospitais a fim de a ambulância tomar uma decisão.

Uma ambulância real não necessariamente utilizaria métodos GET e POST, podendo utilizar no lugar WebSockets ou outros métodos que mantenham o update em tempo real para o paramédico. Esta porta de entrada da aplicação foi importante no desenvolvimento da infraestrutura para realização de testes funcionais dos outros módulos além da verificação de tipos de arquivo como converter o áudio WEBM

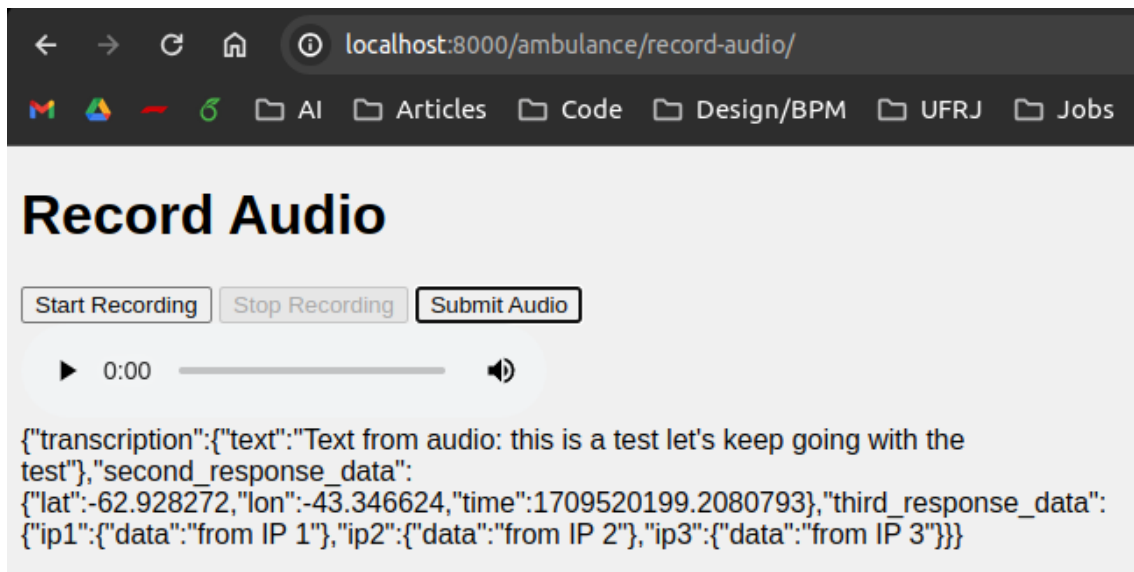


Figura 3.2: Página Web do módulo da ambulância.

gravado pelo navegador para enviar uma requisição POST contendo um arquivo WAV para a API Processadora de Voz. Como utilizamos uma interface Web a ativação do microfone USB conectado ao hardware se dá através da autorização do navegador para gravar áudio. Há muitos métodos de se aprimorar a interface da ambulância em si para um protótipo e produto fechado porém uma vez que esta prova de conceito foca no pipeline CI/CD em si focaremos no Processador de Áudio.

A interface da ambulância em Django possui uma estrutura de arquivos que inclui diversos componentes fundamentais para o desenvolvimento de modelos, páginas web e regras de negócio associadas às URIs do servidor. Esta aplicação Django é extensível, permitindo a adição de múltiplas páginas web e a integração com bancos de dados, o que facilita a criação de funcionalidades adicionais conforme necessário.

A API falsa de GPS é composta apenas por um `Dockerfile` e uma rota padrão para atender a requisições GET, simulando a funcionalidade de um módulo GPS no hardware. A porta 2947 foi escolhida propositalmente, pois é a porta padrão utilizada em sistemas Linux para o GPS daemon. Esta API pode ser expandida no futuro para incluir funcionalidades mais complexas de GPS.

O agregador de requisições, desenvolvido com FastAPI e acessado através do NginX, é um módulo adicional construído como exemplo. Embora o agregador não seja parte central dos módulos de borda, ele ilustra a capacidade do servidor Django de interagir com ele. O módulo serve como uma simulação de um serviço

que, em um cenário real, poderia consultar a web para localizar diversos hospitais. Este repositório foi propositalmente mantido simples, pois seu objetivo é apenas demonstrativo dentro do contexto do projeto.

3.2 A Infraestrutura

A Tabela 3.2 abaixo é uma referência das portas mapeadas para a máquina hospedeira utilizada no projeto.

Tabela 3.4: Tabela de contêineres Docker do projeto.

Porta	Serviço
2947	GPS
5000	Audio Processing
8000	Ambulância
8080	Jenkins
9000	FastAPI (Agregador)
9500	NginX (Agregador)

No repositório há o script `run_local.sh` para criar esta composição inicial do projeto.

O script aciona todos os containers na máquina que está sendo executada o projeto localmente onde o Jenkins funciona como nó mestre e único.

Todos os containers estão descritos em arquivos `Dockerfiles` e `docker-compose.yml`.

A descrição de como o pipeline do Jenkins foi construído está no arquivo `Jenkinsfile`.

O método de download da imagem e criação do container Jenkins está escrito no `README.md`.

Ao executar o Jenkins lhe dará uma senha inicial que deve ser guardada para iniciar o Jenkins através da interface gráfica pelo navegador na porta local 8080. Ao navegar no Jenkins deve-se adicionar uma chave gerada no GitHub chamada GitHub Token que permite o Jenkins fazer consultas ao repositório do GitHub e verificar atualizações.

As informações permanentes do Jenkins estão armazenadas em um volume Docker criado durante a etapa de criação do Jenkins assim como a conexão da rede com a rede local da máquina Linux e o mapeamento do UNIX socket Docker daemon para dentro do contêiner Jenkins. Desta forma o contêiner pode controlar os outros contêineres da aplicação para modificá-los e atualizá-los durante a execução do pipeline.

Ao clonar o repositório, podemos acessar a aplicação na nossa rede local e executá-la da mesma forma do projeto acima.

A decisão de utilizar Jenkins como ferramenta principal após testes e pesquisa com as ferramentas apresentadas no Capítulo 2 foi motivada pela questão deste possuir a maior comunidade de usuários e demanda profissional dentre os pipelines CI/CD apresentados no mercado. Além disso, o Jenkins possui plugins para Docker, Ansible, Terraform e Kubernetes. Desta forma é possível integrar todas as outras ferramentas citadas dentro do pipeline.

O Jenkins está sendo executado como um nó único localmente para exemplificar o que um nó consegue fazer em um pipeline como criar imagens e contêineres de teste, testá-los, excluí-los e iniciar uma nova implantação.

Em relação a ferramentas como Ansible, a implantação teria que partir da nuvem para ambulância de forma que a nuvem guarde uma lista de IP de todas as ambulâncias. Partindo do pressuposto de IP variável a arquitetura de múltiplos nós do Jenkins permite que cada ambulância se conecte com o nó mestre na nuvem para distribuir testes automatizados na nuvem e o nó da ambulância execute apenas a implantação.

A Figura 3.3 mostra pipelines sendo executados com sucesso e logo após a Tabela 3.6 contém a descrição de cada etapa do pipeline mostrado na figura.



	Declarative: Checkout SCM	Timestamp	Update Submodules	Build Dummy GPS API Image	Run Dummy GPS API Container	Wait for Dummy GPS API Service Startup	Test Dummy GPS Service	Build Voice Processing Image	Run Voice Processing Container	Wait for Voice Processing Service Startup	Test Voice Processing Service for Smoke Inhalation	Test Voice Processing Service for Blood Loss	Test Voice Processing Service for Orthopedics	Push Images to Docker Hub	Deploy	Declarative: Post Actions
Average stage times: (Average full run time: ~1min 13s)	763ms	425ms	1s	2s	515ms	5s	324ms	1s	508ms	5s	5s	4s	5s	17s	7s	9s
mar: 04 00:43 1 commit	877ms	376ms	852ms	1s	544ms	5s	324ms	1s	785ms	5s	4s	5s	5s	14s	24s	1s
mar: 04 00:39 1 commit	922ms	465ms	924ms	2s	556ms	5s	302ms	1s	543ms	5s	5s	5s	9s	15s	23s	1s
mar: 04 00:30 No Changes	712ms	456ms	977ms	6s	563ms	5s	332ms	1s	295ms	5s	4s	5s	4s	19s	1s	13s

Figura 3.3: Pipelines executados com sucesso.

Tabela 3.5: Resultados das Execuções de Pipeline na figura 3.3 (Transposta e reduzida).

Etapas	Execução #89	Execução #88
Declarative: Checkout SCM	877ms	922ms
Timestamp	376ms	465ms
Update Submodules	852ms	924ms
Build Dummy GPS API Image	1s	2s
Run Dummy GPS API Container	544ms	556ms
Wait for Dummy GPS API Service Startup	5s	5s
Test Dummy GPS Service	324ms	302ms
Build Voice Processing Image	1s	1s
Run Voice Processing Container	785ms	543ms
Wait for Voice Processing Service Startup	5s	5s
Test Voice Processing Service for Smoke Inhalation	4s	5s
Test Voice Processing Service for Blood Loss	5s	5s
Test Voice Processing Service for Orthopedics	5s	9s
Push images to Docker Hub	14s	15s
Deploy	24s	23s
Post Actions	1s	1s

Tabela 3.6: Etapas do Pipeline CI/CD do Jenkins.

Passo	Explicação
Declarative: Checkout SCM	Este passo realiza o checkout do código-fonte do repositório do sistema de controle de versão (SCM), como Git ou Subversion, para o workspace do Jenkins.
Timestamp	Adiciona um carimbo de data/hora ao log de compilação para rastrear quando a compilação foi iniciada.
Update Submodules	Atualiza quaisquer submódulos Git dentro do repositório para a revisão mais recente.
Build Dummy GPS API Image	Constrói a imagem Docker para o serviço Dummy GPS API com base no Dockerfile fornecido no repositório.
Run Dummy GPS API Container	Inicia um contêiner Docker usando a imagem Dummy GPS API previamente construída.

Tabela 3.6: Etapas do Pipeline CI/CD do Jenkins (Continuação).

Passo	Explicação
Wait for Dummy GPS API Service Startup	Pausa a execução do pipeline até que o serviço Dummy GPS API esteja totalmente iniciado e pronto para aceitar solicitações.
Test Dummy GPS Service	Executa testes automatizados contra o serviço Dummy GPS API para garantir sua funcionalidade.
Build Voice Processing Image	Constrói a imagem Docker para o serviço de Processamento de Voz com base no Dockerfile fornecido no repositório.
Run Voice Processing Container	Inicia um contêiner Docker usando a imagem de Processamento de Voz previamente construída.
Wait for Voice Processing Service Startup	Pausa a execução do pipeline até que o serviço de Processamento de Voz esteja totalmente iniciado e pronto para aceitar solicitações.
Test Voice Processing Service for Smoke Inhalation	Executa testes de fumaça contra o serviço de Processamento de Voz para garantir a funcionalidade básica.
Test Voice Processing Service for Blood Loss	Executa testes adicionais para verificar a funcionalidade específica relacionada a cenários de perda de sangue.
Test Voice Processing Service for Orthopedics	Executa testes adicionais para verificar a funcionalidade específica relacionada a cenários ortopédicos.
Push images to Docker Hub	Faz o upload das imagens Docker construídas para o Docker Hub ou outro registro de contêiner para armazenamento e compartilhamento.

Tabela 3.6: Etapas do Pipeline CI/CD do Jenkins (Continuação).

Passo	Explicação
Deploy	Implanta os serviços construídos no ambiente de destino, como um servidor de desenvolvimento, estágio ou produção.
Declarative: Post Actions	Define quaisquer ações pós-compilação, no caso remover imagens e containers criados pelo pipeline.

Neste caso podemos observar que os pipelines apresentam uma sequência de sucessos ao construir as imagens de Processamento de Voz e GPS Fictício, além dos testes utilizando expressões regulares para verificar se os áudios estão sendo traduzidos para o contexto correto.

No caso de por exemplo, substituir-se a API do Google pelo PocketSphinx que possui uma perda de qualidade de áudio elevada e testarmos o pipeline novamente pode-se verificar a falha no teste do Jenkins ao comparar o texto de saída do PocketSphinx com o contexto esperando através de expressões regulares.

Na Figura 3.4, é ilustrado o cenário em que a API do Google é substituída pelo PocketSphinx. Neste caso, o Jenkins encerra os testes antes da etapa de deploy, pois o primeiro teste com expressões regulares falha ao não conseguir identificar palavras relacionadas à inalação de fumaça no texto processado. A Figura 3.5 exibe o console da execução 91 deste pipeline, onde um comando `curl` executa uma requisição HTTP POST, enviando um áudio relacionado a inalação de fumaça e estresse respiratório. Nota-se que o texto gerado não corresponde ao contexto esperado, falhando na comparação com as expressões regulares pré-definidas. Como resultado, as etapas subsequentes, incluindo os testes de processamento de voz para sangue e trauma ortopédico, o envio de novas imagens de contêineres para o Docker Hub, e o novo deploy, são omitidas. No entanto, o passo final de pós-processamento é sempre executado, independentemente do sucesso ou falha, para garantir a limpeza das dependências específicas desses testes.



Figura 3.4: Pipeline falhou ao testar o processamento de áudio para inalação de fumaça.

Tabela 3.7: Resultados das Execuções de Pipeline na Figura 3.4 (Transposta).

Etapa	Execução #91	Execução #90
Checkout SCM	1s	582ms
Timestamp	387ms	441ms
Update Submodules	881ms	912ms
Build Dummy GPS API Image	1s	2s
Run Dummy GPS API Container	534ms	302ms
Wait for Dummy GPS API Service Startup	5s	5s
Test Dummy GPS Service	343ms	311ms
Build Voice Processing Image	1s	1s
Run Voice Processing Container	292ms	541ms
Wait for Voice Processing Service Startup	5s	5s
Test Voice Processing Service for Smoke Inhalation	12s (failed)	4s
Test Voice Processing Service for Blood Loss	89ms (failed)	4s
Test Voice Processing Service for Orthopedics	40ms (failed)	5s
Push images to Docker Hub	35ms (failed)	15s
Deploy	25ms	14s
Post Actions	13s	13s

```
[Pipeline] { (Test Voice Processing Service for Smoke Inhalation)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ curl -X POST -F file=@/var/jenkins_home/workspace/TCC CI Pipeline/TCC_Voice_Processing/audios/smoke_inhalation_respiratory_distress.wav voice_processing_container:5000/transcribe
% Total    % Received % Xferd  Average Speed   Time    Time     Time
           Dload  Upload   Total   Spent    Left     Speed

  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--   0
100 1283k    0     0 100 1283k      0 1067k  0:00:01  0:00:01 --:--:-- 1068k
100 1283k    0     0 100 1283k      0  582k  0:00:02  0:00:02 --:--:--  582k
100 1283k    0     0 100 1283k      0  400k  0:00:03  0:00:03 --:--:--  400k
100 1283k    0     0 100 1283k      0  305k  0:00:04  0:00:04 --:--:--  305k
100 1283k    0     0 100 1283k      0  246k  0:00:05  0:00:05 --:--:--  246k
100 1283k    0     0 100 1283k      0  206k  0:00:06  0:00:06 --:--:--    0
100 1283k    0     0 100 1283k      0  178k  0:00:07  0:00:07 --:--:--    0
100 1283k    0     0 100 1283k      0  156k  0:00:08  0:00:08 --:--:--    0
100 1283k    0     0 100 1283k      0  139k  0:00:09  0:00:09 --:--:--    0
100 1283k    0     0 100 1283k      0  125k  0:00:10  0:00:10 --:--:--    0
100 1283k    0     0 100 1283k      0  114k  0:00:11  0:00:11 --:--:--    0
100 1283k    0     0 100 1283k      0  105k  0:00:12  0:00:12 --:--:--    0
 99 1283k    0     0 100 1283k      0  101k  0:00:12  0:00:12 --:--:--    0
100 1283k  100    115 100 1283k      9  101k  0:00:12  0:00:12 --:--:--   33
[Pipeline] echo
Curl Output: {"text":"Text from audio: the productive of life and it's numerous numerous live says that and many intervention"}
[Pipeline] error
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test Voice Processing Service for Blood Loss)
Stage "Test Voice Processing Service for Blood Loss" skipped due to earlier failure(s)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test Voice Processing Service for Orthopedics)
Stage "Test Voice Processing Service for Orthopedics" skipped due to earlier failure(s)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Push images to Docker Hub)
Stage "Push images to Docker Hub" skipped due to earlier failure(s)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)

```

Figura 3.5: Pipeline 91 falhou ao testar o processamento de áudio para inalação de fumaça.

Listing 3.1: Execução de comando curl em pipeline Jenkins 89

```
[Pipeline] { (Test Voice Processing Service for Smoke Inhalation)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ curl -X POST -F file=@/var/jenkins_home/workspace/TCC CI Pipeline
    /TCC_Voice_Processing/audios/
    smoke_inhalation_respiratory_distress.wav
    voice_processing_container:5000/transcribe
% Total    % Received % Xferd  Average Speed   Time    Time     Time
           Dload  Upload   Total   Spent    Left     Speed

100 1238k    0     0 100 1238k      0 1068k  0:00:01  0:00:01
    --:--:-- 1068k
100 1238k    0     0 100 1238k      0  582k  0:00:02  0:00:02
    --:--:--  582k
100 1238k    0     0 100 1238k      0  400k  0:00:03  0:00:03
    --:--:--  400k

```



```

100 1238k    0      0  100 1238k    0    305k  0:00:04  0:00:04
  --:--:--  305k
100 1238k    0      0  100 1238k    0    246k  0:00:05  0:00:05
  --:--:--  246k
100 1238k    0      0  100 1238k    0      0  0:00:06  0:00:06
  --:--:--      0
100 1238k    0      0  100 1238k    0      0  0:00:07  0:00:07
  --:--:--      0
100 1238k    0      0  100 1238k    0      0  0:00:08  0:00:08
  --:--:--      0
100 1238k    0      0  100 1238k    0      0  0:00:09  0:00:09
  --:--:--      0
100 1238k    0      0  100 1238k    0      0  0:00:10  0:00:10
  --:--:--      0
100 1238k    0      0  100 1238k    0      0  0:00:11  0:00:11
  --:--:--      0
100 1238k    0      0  100 1238k    0      0  0:00:12  0:00:12
  --:--:--    33
[Pipeline] echo
Curl Output: {"text":"Text from audio: the productive of life and
  it's numerous numerous live says that and many intervention"}
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test Voice Processing Service for Blood Loss)
Stage "Test Voice Processing Service for Blood Loss" skipped due to
  earlier failure(s)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test Voice Processing Service for Orthopedics)
Stage "Test Voice Processing Service for Orthopedics" skipped due
  to earlier failure(s)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Push images to Docker Hub)

```


Listing 3.2: Execução de comando curl em pipeline Jenkins 91

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] sh
+ ./stop_all_except_jenkins.sh
0b46426d5f9
e2833b8bbef7
[Pipeline] sh
+ docker compose -f docker-compose-deploy.yml up --build -d
#0 Building with "default" instance using docker driver

#1 [tcc_ambulance_interface internal] load build definition from
    Dockerfile
#1 transferring dockerfile: 661B done
#1 DONE 0.0s

#2 [tcc_ambulance_interface internal] load metadata for docker.io/
    library/python:3.9-slim
#2 DONE 0.8s

#3 [tcc_ambulance_interface internal] load .dockerignore
#3 transferring context: 2B done
#3 DONE 0.0s

#4 [tcc_ambulance_interface internal] load build context
#4 transferring context: 1.19kB done
#4 DONE 0.0s

#5 [tcc_ambulance_interface 1/8] FROM docker.io/library/python:3.9-
    slim@sha256:
    e0bc011bb55918109921b913fe30160cb8297c570621a450477d44999a792beb
#5 DONE 0.0s

#6 [tcc_ambulance_interface 2/8] WORKDIR /app
#6 DONE 0.0s

#7 [tcc_ambulance_interface 3/8] RUN apt-get update && apt-get
    install -y gcc libpq-dev libmagic1 ffmpeg && rm -rf /var/lib/apt
```

```
    /lists/*
#7 DONE 0.0s

#8 [tcc_ambulance_interface 4/8] COPY requirements.txt /app/
#8 CACHED

#9 [tcc_ambulance_interface 5/8] RUN pip install --no-cache-dir -r
    requirements.txt
#9 CACHED

#10 [tcc_ambulance_interface 6/8] RUN pip install gunicorn
#10 CACHED

#11 [tcc_ambulance_interface 7/8] COPY . /app/
#11 CACHED
```

Os testes realizados no pipeline de CI/CD com Jenkins, utilizando diferentes métodos de implantação do processamento de voz, foram bem sucedidos em termos de aprovar ou reprovar resultados com base na qualidade da transcrição de áudio.

A configuração do pipeline no nó mestre demonstrou como novas imagens podem ser construídas e como testes de transcrição de áudio, utilizando expressões regulares, podem ser realizados de maneira eficiente.

Assim como foi simples configurar a integração entre o GitHub e o Jenkins por meio de um GitHub Token, também é fácil criar múltiplos pipelines para execução em cada ambulância. Um pipeline bem-sucedido no nó mestre pode desencadear a execução de um pipeline em cada ambulância, diretamente em seu nó agente, para iniciar uma nova implantação, caso sejam atendidas regras gerais, como "a ambulância está ociosa e, portanto, pode ser atualizada".

A escalabilidade proporcionada por um único nó mestre na nuvem gerenciando milhares de nós agentes, um em cada ambulância, assegura uma comunicação eficaz entre os testes de integração contínua executados na nuvem e a implantação contínua realizada na borda.

Capítulo 4

Conclusões

Considerando a pesquisa realizada em torno das ferramentas públicas e de código aberto disponíveis no mercado para se realizar este projeto, podemos concluir que para uma prova de conceito o objetivo foi alcançado.

A prova de conceito da ambulância inteligente serviu como um exemplo ideal para explorar o gerenciamento de aplicações executadas na borda, como em hardwares instalados em veículos e dispositivos móveis. O desenvolvimento de uma aplicação web para a ambulância e de um processador de voz, utilizando múltiplos módulos de qualidade variada para processamento de áudio, demonstrou eficazmente como desenvolvedores, trabalhando remotamente, podem manter e atualizar constantemente software em dispositivos de borda, mesmo quando estes estão fisicamente inacessíveis. Além disso, o uso de pipelines CI/CD mostrou-se altamente escalável, permitindo que milhares de ambulâncias possam ser atualizadas simultaneamente de forma eficiente e controlada.

Considerando ferramentas de provisionamento como o Terraform, podemos avaliar que ele é ideal para a criação de instâncias em provedores de nuvem pública. No entanto, para um cenário de projeto na borda, o uso do Terraform pode não ser adequado. O Terraform se baseia na integração com múltiplas APIs e na colaboração entre provedores de nuvem pública e a HashiCorp, empresa responsável pela ferramenta, para garantir que as APIs do Terraform interajam corretamente com os serviços de nuvem. Em um contexto de nuvem privada ou em servidores de borda próprios, como no caso das ambulâncias mencionadas, o uso do Terraform

pode não fazer sentido. Isso ocorre porque esses ambientes geralmente não possuem uma alocação de recursos de hardware definida por software de maneira dinâmica, o que é um dos principais benefícios do Terraform em ambientes de nuvem pública.

Considerando ferramentas de virtualização como Vagrant estas são interessantes para criação de ambientes locais e modos de compatibilidade radicais porém para modularização de serviços são menos eficientes que a construção de imagens de contêineres. A escolha de Vagrant e consequentemente máquinas virtuais ao invés de contêineres em projeto ocorre normalmente na necessidade de compatibilidade de ambientes de desenvolvimento com dependências muito pesadas ou sistemas legado que necessitam compatibilidade.

A premissa inicial de que Ansible por ser uma ferramenta executora de scripts seria a principal ferramenta foi anulada uma vez que o Jenkins possui plugins que o integram com diversos serviços porém o contrário não é verdadeiro. Portanto a estruturação em torno de uma ferramenta com facilidade de integração com outros recursos é vantajosa mesmo que a arquitetura de nós e agentes mestre-escravo possa ser trabalhosa.

A utilização de contêineres Docker para a execução e criação de imagens foi crucial para o sucesso deste projeto. O uso de Docker permite a criação e teste de imagens de maneira consistente e repetível, o que é fundamental para a manutenção e compatibilidade das aplicações, especialmente em cenários que exigem armazenamento persistente e acesso direto ao hardware, como microfones. Esta abordagem garante que o ambiente de execução seja padronizado, facilitando o desenvolvimento, teste e implantação em diferentes infraestruturas.

Criação de scripts ad hoc foi necessária para personalização dos recursos conforme necessário. O pipeline demonstrado possui diversos scripts para execução de determinadas tarefas enquanto a criação do pipeline em si é vantajosa pela organização da estrutura porém mesmo sem o ferramentas como o Jenkins ainda seria possível criar uma sequência de scripts chamando scripts equivalentes ao pipeline criado.

A implementação de um pipeline CI/CD através do Jenkins foi essencial para testar atualizações do módulo de processamento de voz e definir se o pipeline deveria continuar a executar ou não. Arquiteturas com nó mestre como é o caso do Jenkins tendem a ter impactos negativos como falha total em caso de falha do nó principal além de limitação de recursos de hardware de todas as ambulâncias uma vez que cada uma terá um nó para realizar sua implantação de borda.

Uma alternativa que poderia ter sido abordada durante este projeto seria a implantação nos servidores de borda da ambulância utilizando Ansible com SSH para que não seja necessário um nó de Jenkins consumindo recursos do hardware da ambulância.

A aplicação deste projeto foi desenvolvida majoritariamente em Python, uma linguagem conhecida por não ser computacionalmente eficiente. Porém Python continua a ser uma das linguagens mais utilizadas do mundo pela devida a curta curva de aprendizado para começar a se desenvolver e pela extensa comunidade com milhares de bibliotecas e frameworks. Analogamente pode-se dizer que Jenkins é uma ferramenta que não está entre as mais eficientes porém continua sendo amplamente utilizado devido a sua extensa comunidade, milhares de plugins facilitadores e interface gráfica amigável.

Em um projeto mais robusto, ferramentas como o Terraform podem ser utilizadas para gerenciar múltiplas nuvens públicas simultaneamente, facilitando o balanceamento de carga do sistema de testes na nuvem. Além disso, para a implementação de pipelines de CI/CD na nuvem em conjunto com Kubernetes, o uso de ferramentas como o ArgoCD [39] é recomendável. O ArgoCD é uma solução de entrega contínua declarativa projetada para gerenciar e automatizar a implantação de aplicações em clusters Kubernetes, garantindo que o estado desejado das aplicações seja mantido de forma consistente. Também é essencial o uso de ferramentas de monitoramento que garantam a saúde das infraestruturas e aplicações de borda dentro de cada ambulância, enviando dados de integridade do software para a nuvem para ações corretivas e análises contínuas.

O código desenvolvido encontra-se no apêndice deste projeto, assim como no repositório público do GitHub: https://github.com/QIRoss/TCC_Major_Repo.

Como trabalhos futuros, pretende-se implementar uma versão de pipeline de testes para borda sendo executado em uma nuvem privada criada através do **OpenStack**. A criação de diversos pipelines de testes para servidores de teste, homologação e produção executados sob um serviço próprio reduziria os gastos com pagamentos de serviços terceirizados a fim de possuir um produto totalmente independente dos provedores de nuvem públicos.

Referências Bibliográficas

- [1] GOBBI, R., “Qual o nível de maturidade DevOps da sua empresa?”, <https://blog.4linux.com.br/qual-o-nivel-de-maturidade-devops-da-sua-empresa/>, 2019, (Acesso em 19 de Abril 2023).
- [2] LASTER, B., *Jenkins 2: Up and Running - Evolve Your Deployment Pipeline for Next-Generation Automation*. O’Reilly, 2018.
- [3] “KubeEdge architecture example”, <https://kubedge.io/en/docs/kubedge/>, (Acesso em 20 de Abril 2023).
- [4] “K3s architecture example”, <https://docs.k3s.io/architecture>, (Acesso em 20 de Abril 2023).
- [5] RAMÍREZ, S., “FastAPI framework, high performance, easy to learn, fast to code, ready for production”.
- [6] HOLOVATY, A., KAPLAN-MOSS, J., *The Definitive Guide to Django: Web Development Done Right*. 3 ed. Apress, 2019.
- [7] DEJONGHE, D., *NGINX Cookbook Advanced Recipes for High-Performance Load Balancing*. O’Reilly, 2022.
- [8] “Library for performing speech recognition, with support for several engines and APIs, online and offline.”
- [9] “Python library based on Torch”.
- [10] “Python package for music and audio analysis”.
- [11] GRINBERG, M., *Flask Web Development: Developing Web Applications with Python*. O’Reilly Media, 2018.

- [12] PONUTHORAI, P. K., LOELIGER, J., *Version Control with Git*. O'Reilly, 2021.
- [13] MATTHIAS, K., KANE, S. P., *Docker: Up and Running*. O'Reilly, 2015.
- [14] “O que é o DevOps?”
- [15] “Git page”.
- [16] “Groovy-lang page”.
- [17] BURNS, B., *Kubernetes: Up and Running Dive into the Future of Infrastructure*. O'Reilly, 2019.
- [18] HIGGINS, D., *MQTT Essentials*. O'Reilly, 2017.
- [19] BRIKMAN, Y., *Terraform: Up & Running - Writing Infrastructure as Code*. O'Reilly, 2022.
- [20] HASHIMOTO, M., *Vagrant: Up and Running*. O'Reilly, 2013.
- [21] “DevOps Roadmap and descriptions”, <https://roadmap.sh/devops>, (Acesso em 20 de Abril 2023).
- [22] PRATIK PATEL, R. M., *Mastering Cloud-Native Architectures with Datadog*. Packt Publishing, 2021.
- [23] BRAZIL, B., *Prometheus: Up & Running: Infrastructure and Application Performance Monitoring*. O'Reilly, 2020.
- [24] ERIC SALITURO, T. O., *Grafana 8: Enterprise-Level Dashboarding and Monitoring*. Packt Publishing, 2022.
- [25] SHKURO, Y., *Mastering Distributed Tracing: Analyzing performance in microservices and complex systems*. O'Reilly, 2019.
- [26] TED YOUNG, AUSTIN PARKER, B. S., *Observability Engineering: Achieving Production Excellence*. O'Reilly, 2020.
- [27] SNYDER, J., *Implementing Service Level Objectives: A Practical Guide to SLIs, SLOs, and Error Budgets*. O'Reilly, 2021.

- [28] GORMLEY, C., TONG, Z., *Elasticsearch: The Definitive Guide*. O'Reilly Media, 2015.
- [29] TURNBULL, J., *The Logstash Book*. Leanpub, 2015.
- [30] KHAN, R., *Kibana Essentials*. Packt Publishing, 2016.
- [31] INSIGHTS, O. B., “Edge CI/CD Pipeline Part A: Using Jenkins”, <https://www.optisolbusiness.com/insight/edge-cicd-pipeline-part-a-using-jenkins>, 2021, Visited on March 4, 2024.
- [32] SERVICES, A. W., “AWS IoT Core Documentation”, <https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html>, Visited on August 22, 2024.
- [33] MEDA, R., “CI/CD Automation for IoT Workloads in Edge Compute Nodes”, 2021.
- [34] HAT, R., “Red Hat Ansible Edge”.
- [35] VUGT, S. V., *Red Hat Enterprise Linux 8 Administration: Master Linux Server and Virtualization Administration*. John Wiley & Sons, 2020.
- [36] Amazon Web Services, “AWS Greengrass”.
- [37] OpenStack Foundation, “Cloud Edge Computing: Beyond the Data Center”.
- [38] QIRoss, “TCC Major Repo”, https://github.com/QIRoss/TCC_Major_Repo.
- [39] CONTRIBUTORS, A., “ArgoCD: Declarative Continuous Delivery for Kubernetes”, <https://argo-cd.readthedocs.io/>, 2024, Accessed: 2024-08-26.

Apêndice A

Código-fonte da prova de conceito

```
docker-compose-deploy.yml

version: '3.8'

services:
  tcc_ambulance_interface:
    build:
      context: ./TCC_Ambulance_Interface
      dockerfile: Dockerfile
    network_mode: "host"

  tcc_dummy_gps_api:
    image: qiross/tcc-tcc_dummy_gps_api:latest
    network_mode: "host"

  tcc_voice_processing:
    image: qiross/tcc-tcc_voice_processing:latest
    network_mode: "host"

docker-compose.yml

version: '3.8'

services:
  tcc_ambulance_interface:
```

```

    build:
      context: ./TCC_Ambulance_Interface
      dockerfile: Dockerfile
      network_mode: "host"

```

```

tcc_dummy_gps_api:
  build:
    context: ./TCC_Dummy_GPS_API
    dockerfile: Dockerfile
    network_mode: "host"

```

```

tcc_voice_processing:
  build:
    context: ./TCC_Voice_Processing
    dockerfile: Dockerfile
    network_mode: "host"

```

Dockerfile

```

FROM jenkins/jenkins:lts
USER root
RUN apt-get update -qq \
    && apt-get install -qqy apt-transport-https ca-certificates
    curl gnupg2 software-properties-common
RUN curl -fsSL https://download.docker.com/linux/debian/gpg | apt-
    key add -
RUN add-apt-repository \
    "deb [arch=amd64] https://download.docker.com/linux/debian \
    $(lsb_release -cs) \
    stable"
RUN apt-get update -qq \
    && apt-get -y install docker-ce
RUN usermod -aG docker jenkins

```

down_local.sh

```
#!/bin/bash
```

```
docker compose down
```

```
cd TCC_NginX_API_Aggregator && docker compose down && docker stop  
my_jenkins
```

Jenkinsfile

```
pipeline {  
    agent any  
  
    environment {  
        TZ = 'America/Sao_Paulo '  
    }  
  
    stages {  
        stage('Timestamp') {  
            steps {  
                script {  
                    def currentTime = sh(script: 'TZ=America/  
Sao_Paulo date "+%H:%M:%S"', returnStdout:  
true).trim()  
                    echo "Current Time in Brazil: ${currentTime}"  
                }  
            }  
        }  
        stage('Update Submodules') {  
            steps {  
                script {  
                    sh 'git submodule update --init --recursive '  
checkout scm  
                }  
            }  
        }  
        stage('Build Dummy GPS API Image') {  
            steps {  
                dir('TCC_Dummy_GPS_API ') {  
                    sh 'pwd '  
                }  
            }  
        }  
    }  
}
```

```

        sh 'ls -l'
        sh 'docker build -t tcc-tcc_dummy_gps_api .'
    }
}

stage('Run Dummy GPS API Container') {
    steps {
        sh 'docker run -d --network jenkins_network --name
            dummy_gps_container tcc-tcc_dummy_gps_api '
    }
}

stage('Wait for Dummy GPS API Service Startup') {
    steps {
        echo 'Waiting for service to start...'
        sleep time: 5, unit: 'SECONDS'
    }
}

stage('Test Dummy GPS Service') {
    steps {
        sh "curl dummy_gps_container:2947"
    }
}

stage('Build Voice Processing Image') {
    steps {
        dir('TCC_Voice_Processing') {
            sh 'pwd'
            sh 'ls -l'
            sh 'docker build -t tcc-tcc_voice_processing .'
        }
    }
}

stage('Run Voice Processing Container') {
    steps {
        sh 'docker run -d --network jenkins_network --name
            voice_processing_container tcc-
            tcc_voice_processing '
    }
}

stage('Wait for Voice Processing Service Startup') {

```

```

    steps {
        echo 'Waiting for service to start...'
        sleep time: 5, unit: 'SECONDS'
    }
}

stage('Test Voice Processing Service for Smoke Inhalation')
{
    steps {
        script {
            def curlOutput = sh(script: 'curl -X POST -F "
                file=@/var/jenkins_home/workspace/TCC CI
                Pipeline/TCC_Voice_Processing/audios/
                smoke_inhalation_respiratory_distress.wav"
                voice_processing_container:5000/transcribe',
                returnStdout: true).trim()
            echo "Curl Output: ${curlOutput}"
            def regexPattern = /respiratory|smoke|
                inhalation/

            def matchFound = (curlOutput =~ regexPattern).
                find()

            if (matchFound) {
                println "Match found: true"
            } else {
                error "Match not found"
            }
        }
    }
}

stage('Test Voice Processing Service for Blood Loss') {
    steps {
        script {
            def curlOutput = sh(script: 'curl -X POST -F "
                file=@/var/jenkins_home/workspace/TCC CI
                Pipeline/TCC_Voice_Processing/audios/
                blood_loss_transfusion.wav"
                voice_processing_container:5000/transcribe',
                returnStdout: true).trim()

```



```

        echo "Curl Output: ${curlOutput}"
        def regexPattern = /blood|hemorrhage|
            transfusion/

        def matchFound = (curlOutput =~ regexPattern).
            find()

        if (matchFound) {
            println "Match found: true"
        } else {
            error "Match not found"
        }
    }
}

stage('Test Voice Processing Service for Orthopedics') {
    steps {
        script {
            def curlOutput = sh(script: 'curl -X POST -F "
                file=@/var/jenkins_home/workspace/TCC CI
                Pipeline/TCC_Voice_Processing/audios/
                car_accident_legs_arms.wav"
                voice_processing_container:5000/transcribe',
                returnStdout: true).trim()
            echo "Curl Output: ${curlOutput}"
            def regexPattern = /fracture|bone|bones|
                orthopedic|broken|leg|legs|arm|arms/

            def matchFound = (curlOutput =~ regexPattern).
                find()

            if (matchFound) {
                println "Match found: true"
            } else {
                error "Match not found"
            }
        }
    }
}

```

```

stage('Push images to Docker Hub') {
    steps {
        withCredentials([usernamePassword(credentialsId: '0
            f4b64ec-2891-454a-bd68-f4de16081621',
            passwordVariable: 'DOCKER_PASSWORD',
            usernameVariable: 'DOCKER_USERNAME')]) {
            sh "docker login -u $DOCKER_USERNAME -p
                $DOCKER_PASSWORD"

            sh "docker tag tcc-tcc_voice_processing qiross/
                tcc-tcc_voice_processing:latest"
            sh "docker push qiross/tcc-tcc_voice_processing
                :latest"

            sh "docker tag tcc-tcc_dummy_gps_api qiross/tcc
                -tcc_dummy_gps_api:latest"
            sh "docker push qiross/tcc-tcc_dummy_gps_api:
                latest"

        }
    }
}

stage('Deploy'){
    steps {
        sh './stop_all_except_jenkins.sh'
        sh 'docker compose -f docker-compose-deploy.yml up
            --build -d'
        sh 'cd TCC_NginX_API_Aggregator && docker compose
            up -d'
        // sh 'docker compose up --build -d'
    }
}

post {
    always {
        sh 'docker stop dummy_gps_container '
        sh 'docker rm dummy_gps_container '
        sh 'docker stop voice_processing_container '
        sh 'docker rm voice_processing_container '
    }
}

```

```

        echo 'Pipeline execution completed.'
    }
}
}

```

playbook-deploy.yml

```

---
- name: Run Docker Compose
  hosts: localhost
  gather_facts: no

  tasks:
    - name: Run Docker Compose up
      shell: docker compose -f /home/qiross/code/TCC/docker-compose
            .yml up --build -d
      args:
        executable: /bin/bash
        chdir: /home/qiross/code/TCC

```

README.md

```
# TCC_Major_Repo
```

Jenkins first steps:

```

docker image build -t custom-jenkins-docker .
docker run -d -it -p 8080:8080 -p 50000:50000 -v /var/run/docker.
    sock:/var/run/docker.sock -v jenkins_home:/var/jenkins_home --
    network jenkins_network --name my_jenkins custom-jenkins-docker
docker exec my_jenkins cat /var/jenkins_home/secrets/
    initialAdminPassword

```

run_local.sh

```
#!/bin/bash
```

```
docker compose up -d && docker start my_jenkins
```

```
cd TCC_NginX_API_Aggregator && docker compose up -d
```

```
stop_all_except_jenkins.sh
```

```
#!/bin/bash
```

```
jenkins_id=$(docker ps -q -f name=my_jenkins)
```

```
docker ps -q | while read -r container_id; do
    if [ "$container_id" != "$jenkins_id" ]; then
        docker stop "$container_id"
    fi
done
```

```
TCC_Ambulance_Interface/Dockerfile
```

```
FROM python:3.9-slim
```

```
ENV PYTHONDONTWRITEBYTECODE 1
```

```
ENV PYTHONUNBUFFERED 1
```

```
ENV DEBUG False
```

```
WORKDIR /app
```

```
RUN apt-get update \
    && apt-get install -y \
        gcc \
        libpq-dev \
        libmagic1 \
        ffmpeg \
    && rm -rf /var/lib/apt/lists/*
```

```
COPY requirements.txt /app/
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
ENV MAGIC_PATH /usr/lib/x86_64-linux-gnu/libmagic.so.1
```

```
RUN pip install gunicorn
```

```
COPY . /app/
```

```
RUN python manage.py collectstatic --noinput
```

```
EXPOSE 8000
```

```
LABEL name="ambulance_interface"
```

```
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "  
    Ambulance_Interface_Project.wsgi:application"]
```

```
TCC_Ambulance_Interface/README.md
```

```
# TCC_Ambulance_Interface
```

```
docker build -t ambulance_interface_image .
```

```
docker run --name ambulance_interface_app -d -p 8000:8000 --network  
    host ambulance_interface_image
```

```
TCC_Ambulance_Interface/requirements.txt
```

```
asgiref==3.7.2
```

```
backports.zoneinfo==0.2.1
```

```
certifi==2023.11.17
```

```
charset-normalizer==3.3.2
```

```
Django==4.2.9
```

```
djangorestframework==3.14.0
```

```
idna==3.6
```

```
pydub==0.25.1
```

```
python-magic==0.4.27
```

```
pytz==2023.3.post1
```

```
requests==2.31.0
```

```
sqlparse==0.4.4
```

```
typing_extensions==4.9.0
urllib3==2.1.0
```

```
TCC_Ambulance_Interface/Ambulance_Interface/apps.py

from django.apps import AppConfig
```

```
class AmbulanceInterfaceConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "Ambulance_Interface"
```

```
TCC_Ambulance_Interface/Ambulance_Interface/urls.py

from django.urls import path
from .views import record_audio

urlpatterns = [
    path('record-audio/', record_audio, name='record_audio'),
]
```

```
TCC_Ambulance_Interface/Ambulance_Interface/views.py

from django.shortcuts import render
import requests
import magic
from pydub import AudioSegment
from django.http import JsonResponse

def record_audio(request):
    if request.method == 'POST' and request.FILES.get('audio'):
        audio_file = request.FILES['audio']

        audio_type = magic.from_buffer(audio_file.read(), mime=True)

        if audio_type != 'video/webm':
```

```

        return JsonResponse({'error': 'Invalid audio file
                               format. Expected: video/webm'}, status=400)

    audio_file.seek(0)

    audio = AudioSegment.from_file(audio_file, format='webm')
    wav_data = audio.export(format='wav')

    transcribe_url = 'http://127.0.0.1:5000/transcribe'
    files = {'file': ('uploaded_audio.wav', wav_data)}
    response = requests.post(transcribe_url, files=files)

    if response.status_code == 200:
        second_response = requests.get('http://127.0.0.1:2947')

        if second_response.status_code == 200:
            third_response = requests.get('http
                                           ://127.0.0.1:9500')
            if third_response.status_code == 200:
                return JsonResponse({
                    'transcription': response.json(),
                    'second_response_data': second_response.
                        json(),
                    'third_response_data': third_response.json
                        ()
                })
            else:
                return JsonResponse({
                    'transcription': response.json(),
                    'error': 'Failed to fetch data from
                               127.0.0.1:9500'
                }, status=500)
        else:
            return JsonResponse({
                'transcription': response.json(),
                'error': 'Failed to fetch data from
                               127.0.0.1:2947'
            }, status=500)
    else:

```

```

        return JsonResponse({'error': 'Failed to transcribe
                                audio'}, status=500)

    return render(request, 'record_audio.html')

```

TCC_Ambulance_Interface/Ambulance_Interface/templates/record_audio.html

```

<!-- templates/record_audio.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
        scale=1.0">
    <title>Record Audio</title>
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></
        script>
    {% load static %}
    <input type="hidden" id="csrfToken" value="{{ csrf_token }}">
    <script src="{% static 'js/record_audio.js' %}"></script>
    <link rel="stylesheet" type="text/css" href="{% static 'css/
        styles.css' %}">
</head>
<body>
    <h1>Record Audio</h1>
    <div>
        <button id="startRecording">Start Recording</button>
        <button id="stopRecording" disabled>Stop Recording</button>
        <button id="submitAudio" disabled>Submit Audio</button>
    </div>
    <audio controls id="audioPreview"></audio>
    <div id="message"></div>
</body>
</html>

```

TCC_Ambulance_Interface/Ambulance_Interface_Project/asgi.py


```

import os

from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "
    Ambulance_Interface_Project.settings")

application = get_asgi_application()

TCC_Ambulance_Interface/Ambulance_Interface_Project/settings.py
"""
Django settings for Ambulance_Interface_Project project.

Generated by 'django-admin startproject' using Django 4.2.

For more information on this file, see
https://docs.djangoproject.com/en/4.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/4.2/ref/settings/
"""

from pathlib import Path
import os

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/4.2/howto/deployment/
# checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = "django-insecure-9rh!uu!xn8bx)u10xeh#kyuvvvfm@o=jp8ua&
    c*w5@kfg_zgyh"

```

```

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    'rest_framework',
    'Ambulance_Interface',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework.authentication.SessionAuthentication',
    )
}

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

ROOT_URLCONF = "Ambulance_Interface_Project.urls"

TEMPLATES = [
    {

```

```

    "BACKEND": "django.template.backends.django.DjangoTemplates
    ",
    "DIRS": [
        os.path.join(BASE_DIR, 'templates'),
    ],
    "APP_DIRS": True,
    "OPTIONS": {
        "context_processors": [
            "django.template.context_processors.debug",
            "django.template.context_processors.request",
            "django.contrib.auth.context_processors.auth",
            "django.contrib.messages.context_processors.
                messages",
        ],
    },
},
],
]

```

```
WSGI_APPLICATION = "Ambulance_Interface_Project.wsgi.application"
```

```
# Database
```

```
# https://docs.djangoproject.com/en/4.2/ref/settings/#databases
```

```

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}

```

```
# Password validation
```

```

# https://docs.djangoproject.com/en/4.2/ref/settings/#auth-password-validators

```

```

AUTH_PASSWORD_VALIDATORS = [
    {
        "NAME": "django.contrib.auth.password_validation.

```

```

        UserAttributeSimilarityValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.
            MinimumLengthValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.
            CommonPasswordValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.
            NumericPasswordValidator",
    },
]

# Internationalization
# https://docs.djangoproject.com/en/4.2/topics/i18n/

LANGUAGE_CODE = "en-us"

TIME_ZONE = "UTC"

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/4.2/howto/static-files/

STATIC_URL = '/static/'

STATICFILES_DIRS = [
    BASE_DIR / "static",
]

STATIC_ROOT = BASE_DIR / "staticfiles"

```

```
# Default primary key field type
# https://docs.djangoproject.com/en/4.2/ref/settings/#default-auto-field
```

```
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

TCC_Ambulance_Interface/Ambulance_Interface_Project/urls.py

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('ambulance/', include('Ambulance_Interface.urls')),
]

urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
```

TCC_Ambulance_Interface/Ambulance_Interface_Project/wsgi.py

```
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "
    Ambulance_Interface_Project.settings")

application = get_wsgi_application()
```

TCC_Ambulance_Interface/static/css/styles.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f0f0f0;
}
```

TCC_Ambulance_Interface/static/js/record_audio.js

```
let mediaRecorder;
let audioChunks = [];
let csrfToken = document.getElementById('csrfToken').value;

document.addEventListener('DOMContentLoaded', function() {
  navigator.mediaDevices.getUserMedia({ audio: true })
    .then(stream => {
      mediaRecorder = new MediaRecorder(stream);

      mediaRecorder.addEventListener("dataavailable", event
        => {
          audioChunks.push(event.data);
        });

      mediaRecorder.addEventListener("stop", () => {
        const audioBlob = new Blob(audioChunks, { type: '
          audio/wav' });
        const audioUrl = URL.createObjectURL(audioBlob);
        const audioPreview = document.getElementById('
          audioPreview');
        audioPreview.src = audioUrl;
        audioPreview.controls = true;

        const submitButton = document.getElementById('
          submitAudio');
        submitButton.disabled = false;
      });
    });

  document.getElementById('startRecording').addEventListener('
    click', function() {
```

```

    audioChunks = [];

    mediaRecorder.start();
    document.getElementById('startRecording').disabled = true;
    document.getElementById('stopRecording').disabled = false;
});

document.getElementById('stopRecording').addEventListener('
    click', function() {
        mediaRecorder.stop();
        document.getElementById('startRecording').disabled = false;
        document.getElementById('stopRecording').disabled = true;
    });

document.getElementById('submitAudio').addEventListener('click
    ', function() {
        const formData = new FormData();
        const audioBlob = new Blob(audioChunks, { type: 'audio/wav'
            });
        formData.append('audio', audioBlob);

        $.ajax({
            url: '/ambulance/record-audio/',
            type: 'POST',
            data: formData,
            processData: false,
            contentType: false,
            headers: {
                'X-CSRFToken': csrfToken
            },
            success: function(response) {
                document.getElementById('message').innerText = JSON
                    .stringify(response);
            },
            error: function(xhr, status, error) {
                document.getElementById('message').innerHTML = '<p>
                    Error submitting audio!</p>';
            }
        });
    });

```

```
});  
});
```

```
TCC_Dummy_GPS_API/Dockerfile  
  
FROM python:3.9-slim  
  
WORKDIR /app  
  
COPY . /app  
  
RUN pip install fastapi uvicorn  
  
EXPOSE 2947  
  
LABEL name="dummy_gps_api"  
  
CMD ["uvicorn", "dummy_gps_api:app", "--host", "0.0.0.0", "--port",  
    "2947"]
```

```
TCC_Dummy_GPS_API/dummy_gps_api.py  
  
from fastapi import FastAPI  
import time  
  
app = FastAPI()  
  
@app.get("/")  
def read_root():  
    latitude = -62.928272  
    longitude = -43.346624  
    timestamp = time.time()  
  
    report = {  
        'lat': latitude,  
        'lon': longitude,  
        'time': timestamp
```



```

    }

    return report

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=2947)

TCC_Dummy_GPS_API/README.md

# TCC_Dummy_GPS_API

docker build -t dummy_gps_image .
docker run --name dummy_gps_api -d -p 2947:2947 --network host
    dummy_gps_image

```

```

TCC_NginX_API_Aggregator/docker-compose.yml

version: '3'

services:
    nginx:
        build: ./nginx
        ports:
            - "9500:80"
        depends_on:
            - fastapi_aggregator

    fastapi_aggregator:
        build: ./fastapi_aggregator
        ports:
            - 9000

```

```

TCC_NginX_API_Aggregator/fastapi_aggregator/Dockerfile

FROM python:3.9-slim

```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "9000"]
```

```
TCC_NginX_API_Aggregator/fastapi_aggregator/main.py
```

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
async def aggregate_data():
```

```
    data_from_ip1 = {"data": "from IP 1"}
```

```
    data_from_ip2 = {"data": "from IP 2"}
```

```
    data_from_ip3 = {"data": "from IP 3"}
```

```
    aggregated_data = {
```

```
        "ip1": data_from_ip1,
```

```
        "ip2": data_from_ip2,
```

```
        "ip3": data_from_ip3
```

```
    }
```

```
    return aggregated_data
```

```
TCC_NginX_API_Aggregator/fastapi_aggregator/requirements.txt
```

```
fastapi==0.70.0
```

```
uvicorn==0.15.0
```

```
TCC_NginX_API_Aggregator/nginx/Dockerfile
```

FROM nginx

COPY nginx.conf /etc/nginx/nginx.conf

TCC_NginX_API_Aggregator/nginx/nginx.conf

worker_processes 1;

events { worker_connections 1024; }

http {

sendfile on;

upstream fastapi {

server fastapi_aggregator:9000;

}

server {

listen 80;

location / {

proxy_pass http://fastapi;

proxy_set_header Host \$host;

proxy_set_header X-Real-IP \$remote_addr;

proxy_set_header X-Forwarded-For

\$proxy_add_x_forwarded_for;

proxy_set_header X-Forwarded-Proto \$scheme;

}

}

}

TCC_Voice_Processing/Dockerfile

FROM python:3.11-slim

WORKDIR /app

```
COPY requirements.txt /app/
```

```
ENV FLASK_APP=voice_api.py
```

```
ENV FLASK_RUN_HOST=0.0.0.0
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
RUN pip install fastapi uvicorn
```

```
COPY . /app/
```

```
EXPOSE 5000
```

```
CMD ["flask", "run"]
```

TCC_Voice_Processing/README.md

```
# TCC_Voice_Processing
```

```
# To test voice_api in my machine:
```

```
curl -X POST -F "file=@/home/qiross/code/TCC/TCC_Voice_Processing/
  audios/smoke_inhalation_respiratory_distress.wav" http
  ://127.0.0.1:5000/transcribe
```

```
docker build -t voice_api_image .
```

```
docker run --name voice_processing_api -d -p 5000:5000 --network
  host voice_api_image
```

Phrases to test Ambulance API:

"We have a patient involved in a car/motorcycle accident with injuries to the legs/arms. Suspected fractures and lacerations present."

"Patient is experiencing perfusion issues and significant blood loss. Immediate attention and blood transfusion may be required."

"Encountered a case of smoke inhalation. The patient is experiencing respiratory distress and requires urgent medical intervention."

"Multiple trauma patient with injuries to the extremities and possible internal injuries. Requesting trauma team activation upon arrival at the hospital."

"Severe chest pain reported, suspecting cardiac issues. Continuous monitoring of vital signs initiated."

"Patient with head trauma following a fall. Unconscious state observed, and pupils are unequal. Immediate neurological assessment required."

"Found a patient with signs of shock due to extensive burns. Administered fluid resuscitation, but further burn care is needed."

"Transporting a patient with a history of allergic reaction. Administered epinephrine, and the patient is stabilized but requires immediate evaluation."

"Suspected spinal injury in a patient involved in a fall. Immobilization precautions in place, and urgent imaging is recommended."

"Motor vehicle collision with entrapment. Extrication completed, and the patient is conscious but complaining of abdominal pain. Preparing for possible internal injuries."

TCC_Voice_Processing/requirements.txt

blinker==1.7.0

certifi==2023.11.17

charset-normalizer==3.3.2

click==8.1.7

Flask==3.0.1

```

idna==3.6
importlib-metadata==7.0.1
itsdangerous==2.1.2
Jinja2==3.1.3
MarkupSafe==2.1.4
requests==2.31.0
SpeechRecognition==3.10.1
typing_extensions==4.9.0
urllib3==2.1.0
Werkzeug==3.0.1
zipp==3.17.0
PocketSphinx==5.0.3
torch==2.1.2
librosa==0.10.1
transformers==4.37.1

```

TCC_Voice_Processing/sr_google_api.py

```

import speech_recognition as sr

def audio_to_text(audio_file):
    recognizer = sr.Recognizer()

    with sr.AudioFile(audio_file) as source:
        audio_data = recognizer.record(source)

    try:
        text = recognizer.recognize_google(audio_data)
        return "Text from audio: {}".format(text)
    except sr.UnknownValueError:
        return "Google Speech Recognition could not understand audio"
    except sr.RequestError as e:
        return "Could not request results from Google Speech Recognition service; {}".format(e)

if __name__ == "__main__":
    audio_file_path = "audios/blood_loss_transfusion.wav"

```

```
print(audio_to_text(audio_file_path))
```

TCC_Voice_Processing/sr_pocketsphinx.py

```
import speech_recognition as sr

def audio_to_text(audio_file):
    recognizer = sr.Recognizer()

    with sr.AudioFile(audio_file) as source:
        audio_data = recognizer.record(source)

    try:
        text = recognizer.recognize_sphinx(audio_data)
        return "Text from audio: {}".format(text)
    except sr.UnknownValueError:
        return "PocketSphinx could not understand audio"
    except sr.RequestError as e:
        return "Error with PocketSphinx recognizer; {}".format(e)

if __name__ == "__main__":
    audio_file_path = "audios/blood_loss_transfusion.wav"
    print(audio_to_text(audio_file_path))
```

TCC_Voice_Processing/sr_torch_librosa.py

```
import torch
import librosa
from transformers import Wav2Vec2ForCTC, Wav2Vec2Processor

def audio_to_text(audio_file):
    model_name = "facebook/wav2vec2-base-960h"
    processor = Wav2Vec2Processor.from_pretrained(model_name)
    model = Wav2Vec2ForCTC.from_pretrained(model_name)

    audio_input, _ = librosa.load(audio_file, sr=18000, mono=True)
```

```

inputs = processor(audio_input, return_tensors="pt", padding="
    longest", truncation=True, max_length=100000)

with torch.no_grad():
    logits = model(inputs.input_values).logits

predicted_ids = torch.argmax(logits, dim=-1)
transcription = processor.batch_decode(predicted_ids)
text = "Transcription:" + transcription[0]
return text

if __name__ == "__main__":
    audio_file_path = "audios/blood_loss_transfusion.wav"
    print(audio_to_text(audio_file_path))

```

TCC_VoiceProcessing/voice_api.py

```

from flask import Flask, request, jsonify
#from sr_google_api import audio_to_text
from sr_pocketsphinx import audio_to_text
# from torch_librosa_example import audio_to_text

voice_api = Flask(__name__)

@voice_api.route('/transcribe', methods=['POST'])
def transcribe_wav():
    if 'file' not in request.files:
        return jsonify({'error': 'No file part'}), 400

    file = request.files['file']

    if file.filename == '':
        return jsonify({'error': 'No selected file'}), 400

    if file and file.filename.endswith('.wav'):
        try:
            text = audio_to_text(file)
            return jsonify({'text': text}), 200

```



```
        except Exception as e:
            return jsonify({'error': str(e)}), 500
    else:
        return jsonify({'error': 'Invalid file format, must be .wav'
                           '}), 400

if __name__ == '__main__':
    voice_api.run(debug=True)
```